



Software Design Specification

LLM 기반 리뷰 신뢰도 분석 맛집 추천 시스템



성균관대학교 소프트웨어공학개론



Team 1



Design Specification Document

Contents

1 Purpose.....	7
1.1 Readership	7
1.2 Scope.....	7
1.3 Objective.....	9
1.4 Document Structure	9
2 Introduction.....	12
2.1 Objectives	12
2.2 Design Notation and Tools.....	12
2.2.1 Used Tools	12
2.2.2 System Architecture Diagram	13
2.2.3 Context Diagram.....	13
2.2.4 Use Case Diagram.....	13
2.2.5 Sequence Diagram.....	13
2.2.6 Activity Diagram.....	15
2.2.7 Entity Relationship Diagram	15
2.2.8 References.....	15
3 System Architecture - Overall.....	16
3.1 Objectives	16
3.2 System Organization	16
3.2.1 Frontend Web Client.....	18
3.2.2 Backend API Server.....	18
3.2.3 Database	18
3.2.4 LLM/Data Analysis Module	19
3.2.5 External Map API.....	19
3.2.6 System Diagram	19
3.3 Use Case Diagram.....	19
3.4 Architectural Design Decisions.....	21
4 System Architecture - Frontend	22

4.1	Objectives	22
4.2	Frontend Overall Structure.....	22
4.3	Route Design	24
4.4	Page Design	25
4.4.1	CampusSelectPage	25
4.4.2	HomePage.....	25
4.4.3	ListOnlyPage.....	27
4.4.4	RestaurantDetailPage.....	27
4.4.5	Admin Pages.....	27
4.5	Component Design.....	28
4.6	Frontend Data Flow	30
4.6.1	General User Data Flow	30
4.6.2	Admin Data Flow	30
4.7	Kakao Map Integration Design	31
4.8	State Management Design	31
4.9	Error and Fallback Design.....	33
4.10	Frontend Sequence Diagrams.....	33
4.10.1	Campus Selection and Restaurant List Loading	33
4.10.2	Filter-based Recommendation Update	33
4.10.3	Kakao Map Failure and List Fallback.....	34
4.11	Frontend Design Constraints.....	34
4.12	Frontend Future Improvements	36
5	System Architecture - Backend	36
5.1	Objectives	36
5.2	Backend Overall Structure	36
5.3	Backend Components.....	37
5.3.1	Campus and Restaurant Query Module	37
5.3.2	Restaurant Detail Module.....	37
5.3.3	Recommendation Service	39
5.3.4	LLM/Data Import Service	39
5.3.5	Review Credibility Score Handling.....	40

5.3.6 Admin Data Management Module	40
5.3.7 Admin Authentication Module.....	40
5.3.8 Admin Restaurant Management Module.....	40
5.4 Backend Data Flow	42
5.4.1 User Request Flow.....	42
5.4.2 LLM/Data Import Flow	42
5.5 Backend Design Constraints.....	42
5.6 Backend Future Improvements	43
6 Protocol Design.....	45
6.1 Objectives.....	45
6.2 Communication Protocol.....	45
6.3 API Design Principles.....	45
6.4 Current MVP API Endpoints	46
6.5 Request Parameter Design.....	46
6.5.1 Main Request Parameters.....	46
6.6 Restaurant List API.....	48
6.6.1 Campus Restaurant List Request.....	48
6.7 Recommendation API.....	49
6.7.1 Recommendation Request	49
6.7.2 Recommendation Response	51
6.8 Restaurant Detail API.....	52
6.8.1 Restaurant Detail Request	52
6.8.2 Response Example.....	52
6.9 LLM/Data Import Protocol	54
6.10 Admin and Analysis APIs.....	54
6.11 Authentication Protocol.....	56
6.12 Error Response Design	57
6.13 Protocol Design Constraints	59
6.14 Future Improvements.....	60
7 Database Design.....	60
7.1 Objectives.....	60

7.2	Current Database Overview	62
7.3	ER Diagram.....	62
7.4	Table Descriptions.....	63
7.4.1	campuses.....	63
7.4.2	restaurants.....	65
7.4.3	restaurant_summaries.....	66
7.4.4	restaurant_tags.....	66
7.5	Relational Schema.....	66
7.6	LLM/Data Import Schema.....	69
7.9	Future Schema Extension.....	71
7.10	Database Design Constraints	71
8	Testing Plan	72
8.1	Objectives.....	72
8.2	Testing Strategy	74
8.3	Frontend Testing.....	74
8.4	Backend API Testing.....	75
8.5	Database and Data Import Testing.....	77
8.6	Recommendation Testing.....	77
8.7	Review Credibility and LLM/Data Testing.....	78
8.8	Admin and Security Testing	78
8.9	Requirements-based Test Cases.....	80
8.10	User Testing.....	81
8.10.1	Alpha Test.....	81
8.10.2	Beta Test.....	81
8.10.3	Acceptance Test.....	81
8.11	Test Environment	81
9	Development Plan	84
9.1	Objectives.....	84
9.2	Development Environment	84
9.3	Frontend Development Plan	86
9.4	Backend Development Plan.....	86

9.5 Database Development Plan	87
9.6 LLM/Data Development Plan	87
9.6.1 Model and Scoring Overview.....	89
9.6.2 Data Output and Backend Integration	89
9.6.3 Limitations and Future Improvements.....	89
9.7 Configuration and Environment Variables.....	89
9.8 Deployment Plan	90
9.9 Version Control and Collaboration Plan.....	92
9.10 Development Risks and Mitigation	92
9.11 Development Milestones.....	93
10 Supporting Information	93
10.1 Objectives	93
10.2 Glossary	95
10.3 Requirement Traceability.....	95
10.4 MVP Scope Summary.....	96
10.5 Future Extension Items	98
10.6 Legal and Ethical Considerations.....	99
10.7 Document Change Management.....	99
10.8 Document Revision History	101
10.9 Reference Materials.....	101

Purpose

본 문서는 LLM 기반 리뷰 신뢰도 분석 맛집 추천 시스템의 소프트웨어 디자인 명세서이다. 본 문서는 앞서 정의된 기능적·비기능적 요구사항을 실제 구현 가능한 소프트웨어 구조로 변환하기 위해 작성되었다. 따라서 본 문서는 시스템이 제공해야 할 기능을 다시 나열하는데 목적을 두기보다, 프론트엔드, 백엔드 API 서버, 데이터베이스, LLM 분석 모듈, 외부 지도 API가 어떤 구조로 구성되고 어떤 방식으로 상호작용해야 하는지를 정의하는 데 초점을 둔다.

본 시스템은 기존 리뷰 플랫폼에 축적된 식당 리뷰 데이터를 분석하여, 리뷰 이벤트나 협찬성 표현의 영향을 상대적으로 줄이고, 사용자가 식당 정보를 더 빠르게 비교할 수 있도록 지원하는 웹 기반 맛집 추천 시스템이다. 시스템은 리뷰 신뢰도 점수, 식당별 요약 정보, 대표 태그, 추천 점수, 추천 근거를 사용자에게 제공하되, 리뷰의 진위 여부나 식당의 실제 품질을 단정하지 않고 의사결정을 돕는 참고 정보로 제공한다.

본 문서는 개발 과정에서 팀원들이 공통된 설계 기준을 공유하고, 구현 단계에서 발생할 수 있는 모듈 간 불일치와 인터페이스 충돌을 줄이기 위한 기준 문서로 사용된다. 또한 시스템 구현, 통합, 테스트, 유지보수의 기준이 되며, 향후 기능 확장이나 LLM 모델 교체가 필요한 경우 설계 변경의 기준 자료로 활용될 수 있다.

1.1 Readership

본 문서는 다음 독자를 대상으로 작성되었다.

첫째, 본 시스템을 직접 구현하는 개발자를 주요 독자로 한다. 프론트엔드 개발자는 화면 구조, 라우팅 방식, 사용자 입력 흐름, 지도 및 리스트 UI 구성 방식을 이해하기 위해 본 문서를 참고한다. 백엔드/API 개발자는 API 구조, 추천 점수 계산 흐름, 관리자 기능 처리

방식, LLM 분석 모듈과의 연동 방식을 확인한다. 데이터베이스 담당자는 ERD, 테이블 구조, 데이터 관계, 무결성 제약사항을 바탕으로 실제 저장 구조를 설계하고 구현한다. LLM 분석 담당자는 리뷰 신뢰도 스코어링 결과가 백엔드 및 데이터베이스와 어떤 방식으로 연결되는지 확인한다.

둘째, 본 문서는 소프트웨어공학개론 수업의 교수자 및 평가자를 위한 설명 자료로 활용된다. 교수자와 평가자는 요구사항 명세서에서 정의된 기능들이 실제 시스템 구조로 어떻게 변환되었는지, 그리고 프론트엔드·백엔드·데이터베이스·LLM 분석 모듈이 어떤 설계 근거에 따라 구성되었는지 검토할 수 있다.

셋째, 본 문서는 향후 시스템을 수정하거나 확장할 유지보수 담당자를 위한 참고 문서로 사용된다. 유지보수 담당자는 본 문서를 통해 기존 설계 구조, 모듈별 책임 범위, 외부 API 및 데이터베이스 의존성, 향후 확장 가능 지점을 파악할 수 있다.

1.2 Scope

본 디자인 명세서는 LLM 기반 리뷰 신뢰도 분석 맛집 추천 시스템의 소프트웨어 설계 범위를 다룬다. 본 시스템의 초기 서비스 범위는 성균관대학교 자연과학캠퍼스 및 인문사회캠퍼스 인근 식당으로 한정된다. 사용자는 서비스에 접속한 뒤 두 캠퍼스 중 하나를 선택하고, 선택한 캠퍼스 권역에 등록된 식당 정보를 지도와 리스트 형태로 탐색할 수 있다.

MVP 단계에서 본 시스템은 일반 사용자용 식당 탐색 기능, 필터 기반 추천 기능, 식당 상세 정보 제공 기능, 지도 기반 탐색 기능, 관리자 데이터 관리 기능을 중심으로 설계된다. 일반 사용자는 별도의 회원가입 없이 캠퍼스를 선택하고, 음식 종류, 가격대, 분위기, 주차 가능 여부 등의 필터를 적용하여 식당 목록과 추천 근거를 확인할 수 있다. 또한 특정 식당을 선택하면 식당의 기본 정보, AI 요약, 대표 태그, 평균 리뷰 신뢰도 점수, 추천 근거를 확인할 수 있다.

프론트엔드는 React 기반 웹 클라이언트로 구성되며, 캠퍼스 선택 화면, 지도 기반 탐색 화면, 리스트 기반 탐색 화면, 식당 상세 정보 화면, 관리자 화면을 제공한다. 백엔드는 FastAPI 기반 API 서버로 구성되며, 프론트엔드의 요청을 처리하고 데이터베이스에 저장된 식당 정보, 요약 정보, 태그 정보, 신뢰도 점수 정보를 조합하여 JSON 형식으로 반환한다. 데이터베이스는 캠퍼스, 식당, 식당 요약, 식당 태그를 기본 구조로 하며, 현재 구현에서는

관리자 리뷰 업로드와 분석 작업 상태 관리를 위해 리뷰 원문과 분석 작업 테이블까지 포함한다. 다만 리뷰별 세부 분석 결과, 관리자 계정 테이블, 관리자 변경 이력 테이블은 현재 구현 범위에는 포함하지 않고 향후 확장 항목으로 분리한다.

LLM 분석 모듈은 리뷰 텍스트를 기반으로 리뷰 신뢰도 점수를 산출하고, 해당 결과를 식당별 요약 정보 및 추천 점수 계산에 활용할 수 있도록 한다. 다만 사용자 요청 시점마다 실시간으로 LLM을 호출하는 방식은 MVP의 기본 설계가 아니며, 관리자 또는 배치 작업을 통해 사전에 분석된 결과를 데이터베이스에 저장한 뒤 사용자 요청 시 조회하는 방식을 우선한다.

본 디자인 명세서에서는 MVP 단계에서 제외되는 기능도 명확히 구분한다. 실시간 GPS 기반 현재 위치 추천, 전국 단위 식당 추천, 예약·결제·쿠폰 기능, 실시간 웨이팅 정보 제공, 완전한 개인 계정 기반 취향 학습은 초기 설계 범위에 포함하지 않는다. 이러한 기능은 향후 시스템 확장 단계에서 추가 가능한 기능으로 분리한다.

1.3 Objective

본 문서의 주요 목적은 요구사항 명세서에 정의된 기능적·비기능적 요구사항을 실제 구현 가능한 소프트웨어 구조로 변환하는 것이다. 요구사항 명세서가 시스템이 수행해야 할 기능과 제약조건을 정의한다면, 본 디자인 명세서는 해당 요구사항을 어떤 구조와 모듈, 인터페이스, 데이터 흐름으로 구현할 것인지를 정의한다.

이를 위해 본 문서는 다음과 같은 세부 목표를 가진다.

첫째, 시스템의 전체 아키텍처를 정의한다. 본 시스템을 Frontend Web Client, Backend API Server, Database, LLM Analysis Module, Kakao Map API로 구분하고, 각 구성 요소의 책임과 상호작용 방식을 설명한다.

둘째, 프론트엔드와 백엔드 사이의 인터페이스를 정의한다. 캠퍼스 목록 조회, 캠퍼스별 식당 목록 조회, 필터 기반 추천, 식당 상세 조회, 관리자 데이터 관리 등 주요 API의 요청 방식과 응답 형식을 설계한다.

셋째, 데이터베이스 구조와 데이터 흐름을 정의한다. 현재 구현에서는 캠퍼스, 식당, 식당 요약, 식당 태그를 중심으로 사용자용 조회 기능을 구성하고, 관리자 리뷰 업로드와 분석

작업 상태 관리를 위해 리뷰 원문과 분석 작업 테이블을 함께 사용한다. 리뷰별 세부 분석 결과, 다중 관리자 계정, 관리자 변경 이력은 향후 확장 가능한 데이터 구조로 분리한다.

넷째, LLM 분석 결과의 불확실성을 고려한 설계를 제시한다. 리뷰 신뢰도 점수와 AI 요약은 절대적 판단이 아니라 사용자 의사결정을 돕는 참고 정보로 제공하며, 관리자가 부정확한 요약이나 태그를 검토하고 수정할 수 있도록 설계한다.

다섯째, 시스템의 테스트와 유지보수를 위한 기준을 마련한다. 기능 구현뿐 아니라 성능, 신뢰성, 보안성, 유지보수성, 이식성을 고려한 설계 기준을 제시하고, 향후 기능 확장과 모델 교체가 가능하도록 모듈화된 구조를 정의한다.

1.4 Document Structure

본 디자인 명세서는 다음과 같은 구조로 구성된다.

1장 Purpose에서는 본 문서의 목적, 예상 독자, 설계 범위, 세부 목표, 문서 구조를 설명한다.

2장 Introduction에서는 본 문서에서 사용하는 주요 설계 다이어그램, 설계 도구, 참고 자료를 설명한다.

3장 System Architecture - Overall에서는 시스템의 전체 아키텍처와 주요 사용자, 시스템 구성 요소, 데이터 흐름을 설명한다.

4장 System Architecture - Frontend에서는 React 기반 프론트엔드의 라우팅 구조, 화면 구성, 컴포넌트 설계, 지도 API 연동, 오류 및 fallback 처리를 설명한다.

5장 System Architecture - Backend에서는 FastAPI 기반 백엔드 서버의 모듈 구조, 식당 조회, 추천 점수 계산, 관리자 데이터 관리, LLM 분석 작업 연동 방식을 설명한다.

6장 Protocol Design에서는 프론트엔드와 백엔드 사이의 REST API 통신 방식, JSON 요청 및 응답 형식, 인증 및 오류 응답 형식을 정의한다.

7장 Database Design에서는 시스템의 데이터베이스 구조, 주요 엔티티, 테이블 관계, 무결성 제약사항, 분석 결과 저장 구조를 설명한다.

8장 Testing Plan에서는 기능 테스트, 통합 테스트, 릴리스 테스트, 사용자 테스트 계획과 주요 테스트 시나리오를 제시한다.

9장 Development Plan에서는 프론트엔드, 백엔드, 데이터베이스, LLM 분석 환경, 외부 API 설정, 개발 제약조건을 정리한다.

10장 Supporting Information에서는 요구사항 추적표, 참고 자료, 작성 및 수정 이력, 향후 확장 가능 항목을 정리한다.

Introduction

본 장에서는 LLM 기반 리뷰 신뢰도 분석 맛집 추천 시스템의 설계를 설명하기 위해 사용되는 주요 다이어그램, 설계 도구, 표기 방식을 정리한다. 1장에서 정의한 설계 범위를 바탕으로, 본 장은 시스템 구조를 어떤 관점에서 표현할 것인지와 각 다이어그램이 이후 장에서 어떤 역할을 수행하는지를 설명한다.

본 시스템은 프론트엔드 웹 클라이언트, FastAPI 기반 백엔드 API 서버, 데이터베이스, LLM/Data 분석 모듈, Kakao Map API가 결합된 웹 기반 추천 시스템이다. 따라서 단일 다이어그램만으로 전체 설계를 설명하기 어렵기 때문에, 전체 아키텍처, 시스템 경계, 사용자 상호작용, 기능별 처리 흐름, 데이터 구조를 구분하여 표현한다.

2.1 Objectives

본 장의 목적은 시스템 설계를 설명하기 위해 사용할 표현 방식을 정의하는 것이다. 본 시스템은 일반 사용자 화면, 관리자 화면, 백엔드 API, 데이터베이스, LLM/Data 분석 모듈, 외부 지도 API가 결합된 구조를 가지므로, 여러 관점의 다이어그램을 함께 사용하여 설계를 설명한다.

본 문서에서는 전체 시스템 구성을 설명하기 위해 System Architecture Diagram과 Context Diagram을 사용한다. 사용자와 시스템 간의 기능 관계는 Use Case Diagram으로 표현하고, 특정 기능이 실행될 때의 시간 순서에 따른 상호작용은 Sequence Diagram으로 표현한다. 관리자 데이터 처리나 LLM/Data 결과 반영 절차는 Activity Diagram으로 나타내며, 데이터베이스의 논리적 구조는 Entity Relationship Diagram과 Relational Schema로 설명한다.

각 다이어그램은 이후 장에서 상세 설계를 설명하는 기준으로 사용된다. 따라서 본 장에서는 각 다이어그램의 목적과 사용 위치를 간단히 정의하고, 구체적인 시스템 구조와 API, 데이터베이스 설계는 3장 이후에서 상세히 설명한다.

2.2 Design Notation and Tools

본 디자인 명세서에서는 시스템의 구조와 동작을 여러 관점에서 설명하기 위해 설계 도구와 다이어그램을 함께 사용한다. 설계 도구는 다이어그램 작성, UI 흐름 정리, API 명세 확인, 데이터베이스 구조 표현을 위해 사용되며, 각 다이어그램은 이후 장의 상세 설계를 보조하는 역할을 한다.

2.2.1 Used Tools

Tool	Usage
Microsoft PowerPoint	발표 및 문서 삽입용 시스템 구조도, 유스케이스 다이어그램, 시퀀스 다이어그램, 액티비티 다이어그램의 초안을 작성하는 데 사용한다.
draw.io / diagrams.net	시스템 아키텍처, 컴포넌트 구조, ERD 등 구성 요소 간 관계가 복잡한 다이어그램을 정리하는 데 사용한다.
Figma	캠퍼스 선택 화면, 지도 기반 탐색 화면, 리스트 화면, 식당 상세 화면, 관리자 화면의 UI 흐름과 와이어프레임을 작성하는 데 사용한다.
GitHub	프론트엔드·백엔드 코드와 문서 버전 관리, 구현 변경 이력 확인, 설계 문서와 실제 코드 간 차이 검토에 사용한다.
dbdiagram.io / ERD Tool	캠퍼스, 식당, 식당 요약, 식당 태그 등 데이터베이스 테이블 관계를 시각화하는 데 사용한다.
FastAPI Swagger / OpenAPI	백엔드 API의 요청 파라미터, 응답 형식, 오류 응답을 확인하고 프론트엔드와 백엔드 간 인터페이스 불일치를 줄이는 데 사용한다.

이러한 도구는 설계 문서와 실제 구현 코드 사이의 불일치를 줄이고, 프론트엔드·백엔드·데이터베이스·LLM/Data 담당자가 공통된 구조를 기준으로 작업할 수 있도록 지원한다. 특히 본 프로젝트는 프론트엔드, 백엔드 API 서버, 데이터베이스, LLM/Data 분석 결과, 외부 지도 API가 결합된 구조이므로, 각 관점에 맞는 다이어그램을 구분하여 사용한다.

2.2.2 System Architecture Diagram

System Architecture Diagram은 본 시스템의 전체 구성 요소와 계층 구조를 표현하는 다이어그램이다. 본 시스템은 크게 Frontend Web Client, Backend API Server, Database, LLM/Data Analysis Module, External Map API로 구성된다.

일반 사용자의 요청은 프론트엔드 웹 클라이언트를 통해 백엔드 API 서버로 전달된다. 백엔드는 데이터베이스에 저장된 캠퍼스 정보, 식당 기본 정보, 식당 요약 정보, 식당 태그, 평균 리뷰 신뢰도 점수를 조합하여 JSON 형태로 반환한다. 프론트엔드는 이 응답을 지도 마커, 식당 리스트, 식당 상세 정보, 추천 근거 형태로 사용자에게 표시한다.

LLM/Data 분석 또는 데이터 처리 흐름은 일반 사용자 요청 흐름과 분리하여 표현한다. 사용자 요청 시점마다 LLM을 실시간 호출하지 않고, 사전에 생성된 LLM/Data 결과를 백엔드가 import하여 데이터베이스에 저장한 뒤 조회하는 구조를 기본으로 한다. 이 다이어그램은 3장 System Architecture - Overall에서 제시한다.

2.2.3 Context Diagram

Context Diagram은 시스템과 외부 주체 사이의 관계를 표현하는 다이어그램이다. 본 문서에서 시스템 경계는 프론트엔드 웹 클라이언트, 백엔드 API 서버, 데이터베이스, LLM/Data 분석 모듈을 포함하는 서비스 전체로 정의한다. 따라서 Context Diagram에서는 시스템 외부의 사용자와 외부 서비스를 중심으로 관계를 표현한다.

본 시스템의 주요 외부 주체는 일반 사용자, 데이터 관리자, 시스템 관리자 및 개발자, Kakao Map API, 외부 리뷰 데이터 또는 LLM/Data 처리 결과이다. 일반 사용자는 웹 클라이언트를 통해 캠퍼스를 선택하고, 식당 정보를 탐색하며, 필터 조건을 입력하고, 추천 결과와 식당 상세 정보를 확인한다. 데이터 관리자는 관리자 화면을 통해 식당 정보, 리뷰 데이터, LLM/Data 결과, 식당 요약 및 태그를 관리한다. 시스템 관리자 및 개발자는 서버 설정, 데이터베이스, 외부 API Key, 모델 연동, 로그 및 장애 대응을 관리한다.

Kakao Map API는 지도 렌더링과 식당 위치 마커 표시를 위해 사용되는 외부 서비스이다. LLM/Data 처리 결과는 리뷰 신뢰도 점수, 식당 요약, 태그 데이터의 원천이 되며, 백엔드 import 또는 관리자 검토를 통해 데이터베이스에 반영된다. 이 다이어그램은 3장 System Architecture - Overall에서 제시한다.

2.2.4 Use Case Diagram

Use Case Diagram은 시스템에서 제공해야 하는 주요 기능과 각 사용자 유형의 관계를 나타내는 다이어그램이다. 본 시스템의 사용자는 크게 일반 사용자, 데이터 관리자, 시스템 관리자 및 개발자로 구분된다.

일반 사용자는 별도의 로그인 없이 캠퍼스 선택, 지도 기반 식당 탐색, 리스트 기반 식당 탐색, 필터 기반 추천, 식당 상세 정보 조회, 추천 근거 확인 기능을 사용할 수 있다. 데이터 관리자는 관리자 영역을 통해 식당 정보와 LLM/Data 결과를 관리하고, 리뷰 CSV 업로드, 분석 작업 상태 확인, 식당 요약 및 태그 검토 기능을 수행한다. 시스템 관리자 및 개발자는 서버 설정, 데이터베이스 관리, 외부 API Key 관리, 로그 확인, 배포 및 장애 대응을 담당한다.

Use Case Diagram은 요구사항 명세서의 기능 요구사항이 어떤 사용자 유형과 연결되는지 보여주며, 이후 테스트 계획에서 각 기능별 테스트 시나리오를 구성하는 기준으로 사용된다. 이 다이어그램은 3장 Use Case Diagram 절에서 제시한다.

2.2.5 Sequence Diagram

Sequence Diagram은 특정 기능이 실행될 때 사용자, 프론트엔드, 백엔드 API 서버, 데이터베이스, 외부 API 또는 LLM/Data 분석 모듈이 시간 순서에 따라 어떤 메시지를 주고받는지 표현하는 다이어그램이다.

본 문서에서는 세 가지 주요 시나리오를 중심으로 Sequence Diagram을 사용한다. 첫째, 캠퍼스 선택 후 식당 목록을 조회하는 흐름이다. 둘째, 사용자가 필터 조건을 변경했을 때 추천 결과가 갱신되는 흐름이다. 셋째, 지도 API 로딩 실패 시 리스트 기반 fallback 화면으로 전환되는 흐름이다.

Sequence Diagram은 프론트엔드 관점의 화면 전환, 상태 변경, API 호출 시점을 설명하는데 사용한다. 상세 API 요청 및 응답 형식은 6장 Protocol Design에서 설명하므로, Sequence Diagram은 4장 Frontend Sequence Diagrams에서 사용자 흐름 중심으로 제시한다.

2.2.6 Activity Diagram

Activity Diagram은 특정 기능이 여러 단계의 작업으로 이루어질 때, 각 단계의 흐름과 분기 조건을 표현하는 데 사용된다. 본 시스템에서는 관리자 데이터 관리 흐름과 LLM/Data 분석 결과 반영 흐름을 설명하기 위해 Activity Diagram을 사용한다.

MVP 및 현재 구현 단계의 데이터 반영 흐름은 식당 기본 정보, 식당별 요약 정보, 식당별 태그 정보, 리뷰 CSV 또는 분석 작업 상태 정보를 백엔드가 검증하고 데이터베이스에 저장하는 방식으로 구성된다. 저장된 데이터는 일반 사용자 화면에서 지도 마커, 식당 리스트, 상세 정보, 추천 근거로 조회된다.

이 다이어그램은 LLM/Data 처리나 관리자 데이터 import 작업이 일반 사용자 요청 시점에 실시간으로 수행되는 것이 아니라, 사전에 수행되어 데이터베이스에 저장된 뒤 사용자 요청 시 조회되는 구조임을 보여주기 위해 사용된다. 필요한 경우 5장 Backend Data Flow에서 제시한다.

2.2.7 Entity Relationship Diagram

Entity Relationship Diagram은 시스템에서 사용하는 데이터 엔티티와 그 관계를 표현하는 다이어그램이다. 본 문서에서는 현재 구현된 데이터베이스 구조와 향후 확장 가능한 데이터베이스 구조를 구분하여 ER Diagram을 작성한다.

현재 구현된 데이터베이스는 사용자용 식당 탐색 기능을 위한 네 개의 핵심 테이블과 관리자 연동을 위한 두 개의 보조 테이블로 구성된다. 사용자 기능의 핵심 테이블은 Campus, Restaurant, RestaurantSummary, RestaurantTag이며, 관리자 리뷰 업로드와 분석 작업 상태 관리를 위해 Review와 AnalysisJob도 함께 사용한다.

향후 확장 단계에서는 ReviewAnalysis, RestaurantAnalysis, AdminUser, AdminChangeLog와 같은 엔티티를 추가할 수 있다. ReviewAnalysis는 리뷰별 신뢰도 점수와 분석 결과를 저장하고, RestaurantAnalysis는 식당 단위의 리뷰 신뢰도 통계를 저장한다. AdminUser와 AdminChangeLog는 관리자 계정과 데이터 수정 이력을 관리한다.

ER Diagram은 7장 Database Design에서 상세히 제시하며, 각 테이블의 Primary Key, Foreign Key, 주요 속성, 무결성 제약사항을 함께 설명한다.

2.2.8 References

본 장과 이후 설계 장에서는 다음 자료를 참고한다.

[1] Team 1, Software Requirements Specification: LLM 기반 리뷰 신뢰도 분석 맛집 추천 시스템

[2] Team 1, Frontend Source Code: SKKU-gourmap-main

[3] Team 1, Backend Source Code and Database Schema Documentation

[4] Team 1, AI Fine-tuning and Review Credibility Scoring Design Report

[5] Kakao Developers, Kakao Map API Documentation

[6] FastAPI Official Documentation

[7] SQLAlchemy Documentation

[8] PostgreSQL Documentation

[9] Hugging Face Documentation

[10] SK Telecom, A.X 4.0 / A.X Light Model Documentation

[11] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications

System Architecture - Overall

3.1 Objectives

본 장에서는 LLM 기반 리뷰 신뢰도 분석 맛집 추천 시스템의 전체 시스템 아키텍처를 정의한다. 본 시스템은 일반 사용자와 관리자 사용자가 접근하는 Frontend Web Client, 요청 처리와 추천 점수 계산을 담당하는 Backend API Server, 식당 정보와 분석 결과를 저장하는 Database, 리뷰 신뢰도 분석 및 데이터 처리 결과를 제공하는 LLM/Data Analysis Module, 지도 기반 탐색을 지원하는 Kakao Map API로 구성된다.

본 장의 목적은 각 구성 요소의 책임과 상호작용 방식을 명확히 구분하는 것이다. 특히 일반 사용자 요청 흐름과 관리자 또는 배치 기반 데이터 반영 흐름을 분리하여, 사용자 화면에서는 사전에 저장된 식당 정보, 요약, 태그, 평균 리뷰 신뢰도 점수를 빠르게 조회할 수 있도록 설계한다.

본 장에서는 전체 시스템 구성, 주요 모듈의 책임, 사용자 유형별 Use Case, 핵심 아키텍처 설계 결정을 설명한다. 세부 프론트엔드 구조는 4장, 백엔드 모듈 구조는 5장, API 형식은 6장, 데이터베이스 구조는 7장에서 상세히 다룬다.

3.2 System Organization

본 시스템은 웹 기반 클라이언트-서버 구조를 따른다. 사용자는 별도의 프로그램 설치 없이 웹 브라우저를 통해 서비스에 접근하며, 프론트엔드는 사용자 입력을 받아 백엔드 API 서버에 요청을 전달한다. 백엔드 서버는 데이터베이스에 저장된 식당 정보, 식당 요약, 식당 태그, 평균 리뷰 신뢰도 점수를 조회하거나 추천 점수를 계산한 뒤, 프론트엔드에 JSON 응답을 반환한다.

지도 기반 탐색 기능은 Kakao Map API를 통해 제공되며, 리뷰 신뢰도 분석 및 식당 요약·태그 데이터는 LLM/Data 처리 결과 또는 관리자 검토 결과로 데이터베이스에

저장된다. 일반 사용자 요청 시점에는 LLM을 실시간으로 호출하지 않고, 사전에 저장된 분석 결과를 조회하는 방식을 기본으로 한다.

전체 시스템은 Frontend Web Client, Backend API Server, Database, LLM/Data Analysis Module, External Map API의 다섯 구성 요소로 나뉜다.

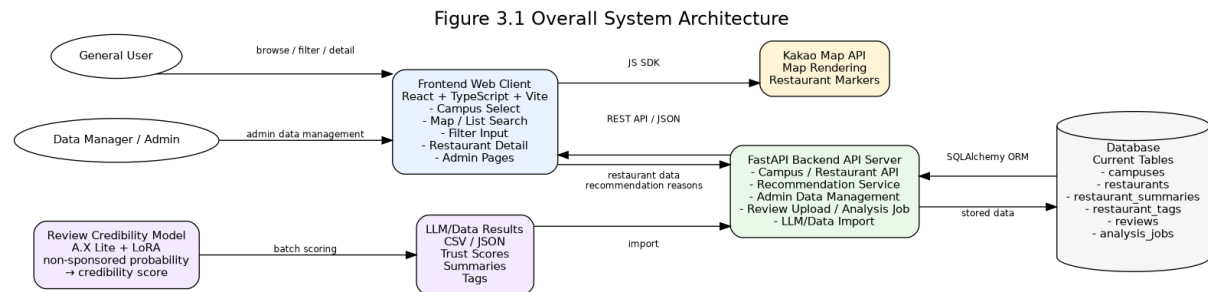


Figure 3.1 Overall System Architecture

Figure 3.1은 본 시스템의 전체 아키텍처를 나타낸다. 일반 사용자는 프론트엔드 웹 클라이언트를 통해 캠퍼스 선택, 식당 탐색, 필터 적용, 식당 상세 조회를 수행한다. 백엔드는 프론트엔드 요청을 처리하고, 데이터베이스에 저장된 식당 정보, 요약, 태그, 평균 리뷰 신뢰도 점수를 조합하여 JSON 응답을 반환한다. LLM/Data 결과는 사용자 요청 시점에 실시간 생성되지 않고, 사전 처리 후 백엔드 import 과정을 통해 데이터베이스에 저장된다.

3.2.1 Frontend Web Client

Frontend Web Client는 일반 사용자와 관리자 사용자가 시스템과 상호작용하는 웹 화면을 제공한다. 일반 사용자 화면은 캠퍼스 선택, 지도 기반 탐색, 리스트 기반 탐색, 필터 기반 추천, 식당 상세 조회 흐름을 제공한다. 관리자 화면은 식당 정보, 리뷰 데이터, LLM/Data 결과, 식당 요약 및 태그를 관리하는 내부 관리 흐름을 제공한다.

프론트엔드는 URL의 `campus_slug`를 기준으로 사용자가 선택한 캠퍼스를 식별하며, 백엔드 API로부터 받은 식당 정보, 요약, 태그, 평균 리뷰 신뢰도 점수, 추천 점수, 추천 근거를 지도 마커와 리스트 카드, 상세 정보 화면에 표시한다. 세부 라우팅 구조, 컴포넌트 설계, 상태 관리 방식은 4장 Frontend Architecture에서 설명한다.

3.2.2 Backend API Server

Backend API Server는 프론트엔드 요청을 처리하고, 데이터베이스와 LLM/Data 결과 사이의 중간 계층으로 동작한다. 백엔드는 캠퍼스 및 식당 조회, 식당 상세 정보 제공, 필터 기반 추천 점수 계산, 관리자 데이터 관리, 리뷰 업로드 및 분석 작업 상태 관리 기능을 API 형태로 제공한다.

일반 사용자 요청에서는 데이터베이스에 저장된 식당 정보, 식당 요약, 식당 태그, 평균 리뷰 신뢰도 점수를 조회하여 JSON 응답을 반환한다. 추천 요청이 들어오면 백엔드는 사용자가 선택한 필터 조건과 식당의 카테고리, 가격대, 태그, 평균 리뷰 신뢰도 점수를 바탕으로 추천 점수와 추천 근거를 계산한다.

백엔드 내부 모듈 구조, 추천 점수 계산 방식, LLM/Data import 흐름, 오류 처리 방식은 5장 Backend Architecture에서 상세히 설명한다.

3.2.3 Database

Database는 시스템의 주요 데이터를 저장하고, 백엔드 API 서버가 사용자 요청을 처리할 때 필요한 정보를 제공하는 핵심 저장소이다. 사용자 기능의 핵심 데이터 구조는 Campus, Restaurant, RestaurantSummary, RestaurantTag로 구성된다. 이 구조는 캠퍼스 기반 식당 목록 조회, 지도 마커 표시, 식당 상세 정보 제공, 필터 기반 추천 점수 계산을 지원한다.

현재 구현에서는 관리자 리뷰 업로드와 분석 작업 상태 관리를 위해 Review와 AnalysisJob도 함께 사용한다. 다만 리뷰별 세부 분석 결과, 식당 단위 분석 통계, 다중 관리자 계정, 관리자 변경 이력은 향후 확장 가능한 데이터 구조로 분리한다.

데이터베이스의 상세 ERD, 테이블 정의, Primary Key와 Foreign Key, 무결성 제약사항은 7장 Database Design에서 설명한다.

3.2.4 LLM/Data Analysis Module

LLM/Data Analysis Module은 리뷰 신뢰도 분석과 식당 정보 재구성을 지원하는 구성 요소이다. 현재 AI 파트에서는 리뷰 텍스트를 기반으로 비협찬성 리뷰일 가능성을 확률값으로 산출하고, 이를 100점 기준의 리뷰 신뢰도 점수로 환산한다.

리뷰 신뢰도 점수는 리뷰의 진짜 또는 가짜 여부를 확정하는 값이 아니라, 리뷰 이벤트 또는 협찬성 표현의 영향을 상대적으로 적게 받았을 가능성을 나타내는 참고 지표이다. 따라서

사용자 화면과 API 응답에서는 "가짜 리뷰", "조작 리뷰"와 같은 단정적 표현을 사용하지 않는다.

MVP 및 현재 구현 흐름에서는 사용자 요청마다 LLM을 실시간 호출하지 않는다. 대신 LLM/Data 처리 결과를 CSV 또는 JSON 형태로 준비하고, 백엔드가 이를 import하거나 관리자 검토를 거쳐 데이터베이스에 저장한 뒤 사용자 요청 시 조회하는 방식을 우선한다.

3.2.5 External Map API

본 시스템은 지도 기반 식당 탐색 기능을 제공하기 위해 Kakao Map API를 사용한다. 프론트엔드는 선택된 캠퍼스의 중심 좌표를 기준으로 지도를 표시하고, 백엔드 API에서 조회한 식당 좌표를 지도 위에 마커로 표시한다.

Kakao Map API는 지도 렌더링과 마커 표시를 담당하지만, 추천 결과와 식당 요약 정보의 원천은 백엔드 API 응답이다. 따라서 지도 마커와 리스트 카드는 동일한 식당 데이터 원천을 사용해야 하며, 필터 조건이 변경될 경우 지도 마커와 리스트 결과가 함께 갱신되어야 한다.

외부 지도 API 장애 또는 로딩 실패가 발생하더라도 식당 탐색 기능이 완전히 중단되지 않도록 리스트 기반 탐색 화면을 대체 경로로 제공한다. 세부 지도 연동 방식은 4장 Frontend Architecture에서 설명한다.

3.2.6 System Diagram

본 절에서는 시스템과 외부 주체 사이의 관계를 Context Diagram으로 정리한다. 전체 내부 구성 요소의 연결 구조는 Figure 3.1에서 제시하였으며, Figure 3.2는 시스템 경계와 외부 의존성을 중심으로 나타낸다.

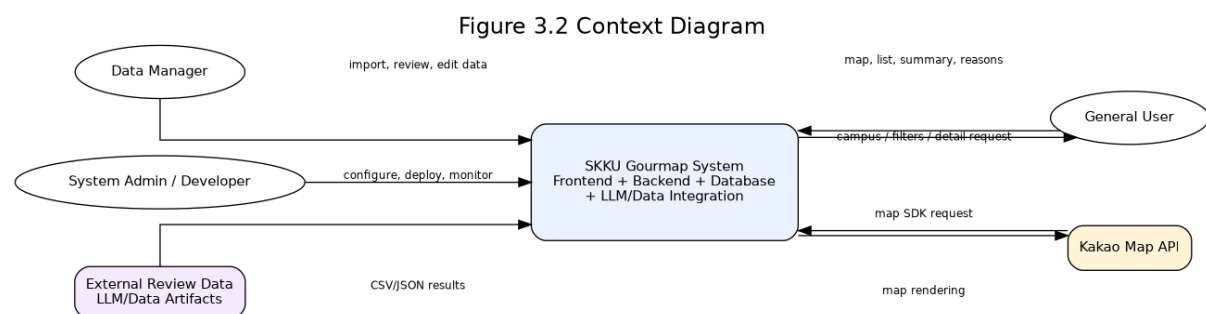


Figure 3.2 Context Diagram

Figure 3.2는 시스템과 외부 주체의 관계를 나타낸다. 일반 사용자는 식당 정보를 탐색하고 추천 결과를 확인하며, 데이터 관리자는 식당 정보, 리뷰 데이터, LLM/Data 결과를 관리한다. 시스템 관리자 및 개발자는 서버 설정, 데이터베이스, 외부 API Key, 로그 및 배포 환경을 관리한다. Kakao Map API는 지도 렌더링과 식당 마커 표시를 위한 외부 서비스로 사용된다.

3.3 Use Case Diagram

Use Case Diagram은 사용자 유형별로 시스템이 제공하는 주요 기능을 표현한다. 본 시스템의 사용자는 일반 사용자, 데이터 관리자, 시스템 관리자 및 개발자로 구분된다.

일반 사용자는 별도의 로그인 없이 캠퍼스 선택, 지도 기반 탐색, 리스트 기반 탐색, 필터 기반 추천, 식당 상세 정보 조회, 추천 근거 확인 기능을 사용한다. 데이터 관리자는 관리자 영역을 통해 식당 정보와 LLM/Data 결과를 관리하고, 리뷰 업로드, 분석 작업 상태 확인, 식당 요약 및 태그 검토 기능을 수행한다. 시스템 관리자 및 개발자는 서버 설정, 데이터베이스, 외부 API Key, 로그, 배포 및 장애 대응을 담당한다.

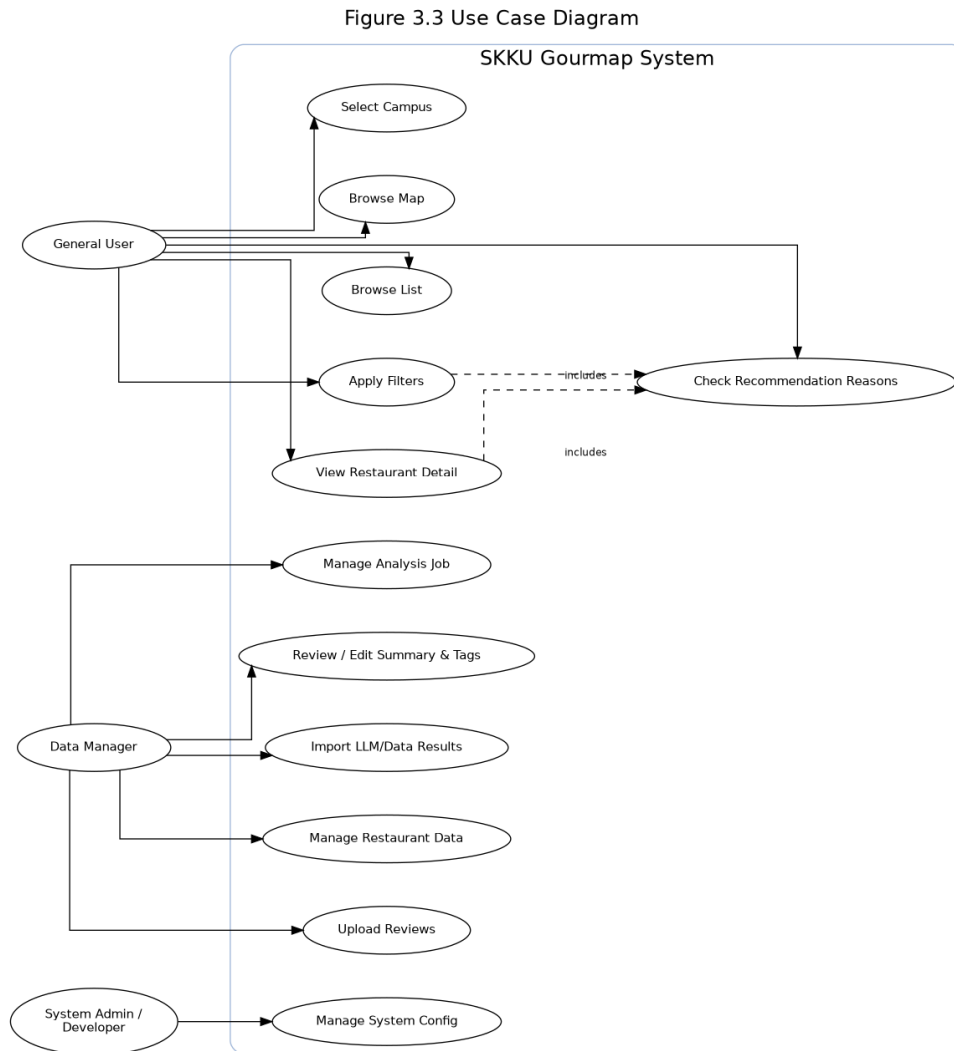


Figure 3.3 Use Case Diagram

Figure 3.3은 사용자 유형별 주요 Use Case를 나타낸다. 일반 사용자는 캠퍼스 선택, 지도 및 리스트 탐색, 필터 적용, 식당 상세 조회, 추천 근거 확인 기능을 사용한다. 데이터 관리자는 LLM/Data 결과 반영, 식당 정보 관리, 리뷰 업로드, 분석 작업 상태 관리, 요약 및 태그 검토 기능을 수행한다. 시스템 관리자 및 개발자는 시스템 설정과 운영 관리를 담당한다.

3.4 Architectural Design Decisions

본 시스템의 전체 아키텍처는 다음 설계 결정을 기반으로 한다.

첫째, 클라이언트-서버 구조를 사용한다. 프론트엔드는 사용자 화면과 입력 처리를 담당하고, 백엔드는 데이터 조회, 추천 점수 계산, 관리자 데이터 관리, LLM/Data 결과 반영을 담당한다. 이를 통해 UI 변경과 서버 로직 변경을 분리한다.

둘째, 일반 사용자 요청과 LLM/Data 분석 흐름을 분리한다. 리뷰 신뢰도 분석과 식당별 요약·태그 데이터 생성은 처리 시간이 길고 비용이 발생할 수 있으므로, 사용자 요청 시점에 실시간으로 수행하지 않는다. 대신 사전에 생성된 분석 결과를 데이터베이스에 저장하고, 일반 사용자 요청에서는 저장된 결과를 빠르게 조회한다.

셋째, 현재 구현 범위를 명확히 구분한다. 사용자 기능의 핵심 데이터는 캠퍼스, 식당, 식당 요약, 식당 태그를 중심으로 구성하고, 관리자 리뷰 업로드와 분석 작업 상태 관리를 위해 리뷰와 분석 작업 데이터를 함께 사용한다. 리뷰별 세부 분석 결과, 식당 단위 분석 통계, 다중 관리자 계정, 관리자 변경 이력은 향후 확장 구조로 분리한다.

넷째, 지도 기반 탐색과 리스트 기반 탐색을 함께 제공한다. 지도는 식당 위치를 직관적으로 보여주는 데 유용하지만, 외부 API 장애나 모바일 환경에서는 사용성이 제한될 수 있다. 따라서 리스트 기반 탐색을 함께 제공하여 지도 API 실패 상황에서도 핵심 식당 탐색 기능을 유지한다.

다섯째, 추천 결과는 설명 가능한 형태로 제공한다. MVP 단계에서는 식당 카테고리, 가격대, 태그, 평균 리뷰 신뢰도 점수를 활용한 규칙 기반 추천 방식을 사용한다. 추천 API는 점수뿐 아니라 추천 근거를 함께 반환하여 사용자가 추천 결과를 이해할 수 있도록 한다.

여섯째, LLM 분석 결과는 참고 지표로 제공한다. 리뷰 신뢰도 점수는 리뷰의 실제 진위 여부를 확정하는 값이 아니므로, 사용자 화면에서는 단정적 표현을 피한다. 또한 데이터 관리자가 요약과 태그를 검토하거나 수정할 수 있는 구조를 둔다.

일곱째, 향후 확장을 고려한 모듈화 구조를 사용한다. 프론트엔드, 백엔드, 데이터베이스, LLM/Data 분석 모듈, 외부 지도 API 연동을 분리하여 서비스 지역 확장, 추천 알고리즘 변경, LLM 모델 교체, 관리자 기능 고도화에 대응할 수 있도록 한다.

System Architecture - Frontend

4.1 Objectives

본 장에서는 LLM 기반 리뷰 신뢰도 분석 맛집 추천 시스템의 프론트엔드 아키텍처를 설명한다. 프론트엔드는 일반 사용자와 관리자 사용자가 직접 상호작용하는 웹 클라이언트이며, 사용자의 입력을 받아 백엔드 API 서버에 요청을 전달하고, 서버로부터 받은 식당 정보, 식당 요약, 태그, 평균 리뷰 신뢰도 점수, 추천 점수, 추천 근거를 화면에 표시한다.

본 장의 목적은 프론트엔드 라우팅 구조, 페이지 구성, 주요 컴포넌트, 데이터 요청 흐름, Kakao Map API 연동 방식, 상태 관리 방식, 오류 및 fallback 설계를 정의하는 것이다.

4.2 Frontend Overall Structure

프론트엔드는 사용자 유형에 따라 일반 사용자 영역과 관리자 영역으로 구분된다. 일반 사용자 영역은 캠퍼스 선택 후 식당을 탐색하고 추천 결과를 확인하는 흐름을 중심으로 구성된다. 관리자 영역은 데이터 관리자가 식당 정보, 리뷰 데이터, LLM 분석 결과, 식당 요약 및 태그를 관리하는 흐름을 중심으로 구성된다.

현재 프론트엔드의 주요 구조는 다음과 같다.

Layer	Main Elements	Description
Routing Layer	React Router v6	일반 사용자 라우트와 관리자 라우트를 분리하고, URL의 campus_slug를 통해 캠퍼스 탐색 범위를 식별한다.

Page Layer	CampusSelectPage, HomePage, ListOnlyPage, RestaurantDetailPage, Admin Pages	화면 단위 UI와 주요 데이터 요청 흐름을 담당한다.
Component Layer	KakaoMap, FilterPanel, RestaurantCard, RestaurantList, TagBadge, ReasonChips, Loading, ErrorMessage, EmptyState, ErrorBoundary	지도, 필터, 식당 카드, 추천 근거, 로딩, 오류, 빈 결과 상태를 재사용 가능한 컴포넌트로 분리한다.
Service Layer	apiClient, restaurantService, categoryService	Axios 기반 API 요청과 백엔드 응답의 프론트엔드 타입 변환을 담당한다.
State Layer	TanStack Query, Zustand, React state, URL query parameter	서버 상태, 관리자 인증 상태, 화면 상태, 필터 상태를 역할별로 분리하여 관리한다.
Config Layer	apiConfig, mapConfig, appConfig	API URL, 캠퍼스 좌표, 지도 설정, 앱 설정값을 관리한다.
Type Layer	api types, restaurant types, kakao maps types	백엔드 응답과 프론트엔드 내부 데이터 구조를 TypeScript 타입으로 정의한다.
Hook Layer	useKakaoLoader, useFilterParams, useRestaurants, useDebounce	SDK 로딩, 필터 URL 동기화, 식당 데이터 조회, 입력 지연 처리를 분리한다.

프론트엔드는 백엔드 API 서버와 REST API 및 JSON 형식으로 통신한다. 사용자가 캠퍼스를 선택하거나 필터를 적용하면, 프론트엔드는 선택된 campus_slug와 필터 조건을 백엔드에 전달한다. 백엔드는 해당 조건에 맞는 식당 정보, 식당 요약, 태그, 평균 리뷰 신뢰도 점수, 추천 점수, 추천 근거를 반환하고, 프론트엔드는 이를 지도 마커, 리스트 카드, 상세 정보 화면에 표시한다.

4.3 Route Design

프론트엔드 라우팅은 일반 사용자 라우트와 관리자 라우트로 구분된다. 일반 사용자 라우트는 로그인 없이 접근 가능하며, 캠퍼스 선택, 식당 탐색, 필터 기반 추천, 식당 상세 조회 기능을 제공한다. 관리자 라우트는 관리자 인증 상태를 확인한 뒤 접근 가능한 내부 관리 화면으로 구성된다.

프론트엔드는 URL의 slug 값을 통해 사용자가 선택한 캠퍼스를 식별한다. 예를 들어 /campus/natural은 자연과학캠퍼스 주변 식당 탐색 화면을 의미하고, /campus/humanities는 인문사회캠퍼스 주변 식당 탐색 화면을 의미한다. 백엔드 API도 campus_slug 기반 엔드포인트를 제공하므로, 프론트엔드 라우팅과 API 호출은 동일한 slug 값을 기준으로 연결된다.

Route	Page Component	Description	Related Requirements
/	CampusSelectPage	자연과학캠퍼스 또는 인문사회캠퍼스 선택	FR-01
/campus/:slug	HomePage	선택된 캠퍼스의 지도 + 리스트 기반 식당 탐색	FR-02, FR-03, FR-04, FR-06, FR-07
/campus/:slug/list	ListOnlyPage	지도 로딩 실패 또는 지도 사용이 어려운 상황의 리스트 기반 fallback 탐색	FR-04, FR-16
/restaurants/:id	RestaurantDetailPage	특정 식당의 상세 정보, AI 요약, 추천 근거 확인	FR-05, FR-08
/admin/login	AdminLoginPage	관리자 로그인	FR-12
/admin/dashboard	AdminDashboardPage	관리자 대시보드	FR-13, FR-14, FR-15
/admin/restaurants	AdminRestaurantsPage	식당 정보	FR-13

	ge	등록·수정·삭제 관리	
/admin/reviews	AdminReviewsPage	리뷰 또는 LLM/Data 결과 업로드 및 관리	FR-14
/admin/analyses	AdminAnalysesPage	분석 결과 및 분석 상태 관리	FR-15

4.4 Page design

4.4.1 CampusSelectPage

CampusSelectPage는 사용자가 서비스를 시작할 때 처음 접하는 화면이다. 사용자는 자연과학캠퍼스와 인문사회캠퍼스 중 하나를 선택할 수 있으며, 선택 결과에 따라 해당 캠퍼스의 식당 탐색 화면으로 이동한다. 캠퍼스 목록은 백엔드의 캠퍼스 목록 API 또는 프론트엔드 설정값을 통해 제공될 수 있으며, 최종 구현에서는 캠퍼스 slug, 표시명, 중심 좌표가 백엔드 데이터와 일관되도록 관리한다.

주요 기능은 다음과 같다.

첫째, 서비스의 목적과 사용 흐름을 간단히 안내한다.

둘째, 자연과학캠퍼스와 인문사회캠퍼스를 선택할 수 있는 카드 또는 버튼을 제공한다.

셋째, 선택된 캠퍼스의 slug 값을 기반으로 /campus/:slug 경로로 이동한다.

넷째, 향후 캠퍼스 목록이 백엔드에서 제공될 경우, API 응답을 기반으로 캠퍼스 선택 항목을 동적으로 렌더링할 수 있다.

4.4.2 HomePage

HomePage는 일반 사용자가 식당 정보를 탐색하는 핵심 화면이다. 이 화면은 선택된 campus_slug를 기준으로 식당 목록을 조회하고, 지도 영역과 리스트 영역을 함께 제공한다. 사용자는 지도에서 식당 위치를 확인하고, 리스트에서 식당별 요약, 태그, 추천 점수, 추천 근거를 비교할 수 있다.

필터가 적용되지 않은 경우 프론트엔드는 캠퍼스별 식당 목록 API를 호출하고, 필터가 적용된 경우 필터 기반 추천 API를 호출한다. 백엔드는 추천 점수와 추천 근거가 포함된 식당 목록을 반환하며, 프론트엔드는 이를 리스트 정렬과 지도 마커 표시 상태에 반영한다.

주요 기능은 다음과 같다.

첫째, URL의 campus_slug를 기반으로 현재 선택된 캠퍼스를 식별한다.

둘째, 선택된 캠퍼스 주변 식당 정보를 조회한다.

셋째, 식당의 위치를 Kakao Map 위에 마커로 표시한다.

넷째, 식당 정보를 카드 또는 리스트 형태로 제공한다.

다섯째, 사용자가 필터 조건을 선택하면 추천 결과와 리스트 정렬 순서를 갱신한다.

여섯째, 특정 식당을 선택하면 식당 상세 페이지로 이동한다.

4.4.3 ListOnlyPage

ListOnlyPage는 지도 없이 식당 정보를 리스트 중심으로 제공하는 fallback 화면이다. 이 화면은 Kakao Map API 로딩 실패, 지도 사용이 불편한 모바일 환경, 또는 사용자가 리스트 중심 탐색을 선택한 경우에 사용된다.

ListOnlyPage는 HomePage와 동일한 백엔드 API 응답을 사용해야 하며, 식당 카드에 표시되는 요약, 태그, 추천 점수, 추천 근거도 HomePage의 리스트 영역과 일관되어야 한다. 즉, ListOnlyPage는 별도의 더미 데이터를 사용하지 않고, 동일한 campus_slug와 필터 조건을 기준으로 식당 목록 또는 추천 결과를 조회해야 한다.

주요 기능은 다음과 같다.

첫째, 선택된 캠퍼스의 식당 목록을 리스트 또는 카드 형태로 표시한다.

둘째, 식당별 AI 요약, 대표 태그, 추천 점수, 추천 근거를 제공한다.

셋째, 사용자가 필터 조건을 변경하면 리스트 정렬 결과를 갱신한다.

넷째, 지도 기반 탐색이 실패해도 사용자가 식당 정보를 계속 확인할 수 있도록 한다.

4.4.4 RestaurantDetailPage

RestaurantDetailPage는 사용자가 특정 식당을 선택했을 때 상세 정보를 제공하는 화면이다. 이 화면은 식당명, 주소, 카테고리, 가격대, 위치 정보와 함께 AI 요약, 대표 메뉴, 분위기 요약, 주차 요약, 웨이팅 요약, 평균 리뷰 신뢰도 점수, 태그, 추천 근거를 표시한다.

리뷰 분석 정보가 부족한 식당의 경우에는 분석 결과를 과도하게 추정하지 않고 “분석 정보가 아직 충분하지 않습니다”와 같은 대체 메시지를 제공한다. 리뷰 신뢰도 점수는

식당의 실제 품질이나 리뷰의 진위를 단정하는 값이 아니므로, 사용자 화면에서는 참고 지표로 표현한다.

주요 기능은 다음과 같다.

첫째, 식당명, 주소, 음식 종류, 가격대, 대표 메뉴, 위치 정보를 표시한다.

둘째, 식당별 AI 요약 정보를 제공한다.

셋째, 리뷰 데이터와 LLM/Data 분석 결과를 바탕으로 정리된 핵심 식당 정보를 표시한다.

넷째, 사용자가 선택한 필터 조건이 있는 경우 해당 식당이 추천된 이유를 제공한다.

다섯째, 리뷰 분석 정보가 부족한 경우 "분석 정보가 아직 없습니다"와 같은 대체 메시지를 표시한다.

이 화면은 FR-05 식당 상세 정보 조회와 FR-08 추천 근거 제공에 대응된다.

4.4.5 Admin Pages

관리자 화면은 일반 사용자 화면과 분리된 내부 관리 영역이다. 관리자 사용자는 로그인 후 식당 정보, LLM/Data 결과, 식당 요약 및 태그를 관리할 수 있다. 프론트엔드는 관리자 라우트를 별도로 구성하고, 인증 상태가 없는 사용자가 관리자 화면에 접근하지 못하도록 라우트 보호 구조를 둔다.

MVP 단계에서 관리자 화면은 식당 데이터와 분석 결과를 관리하기 위한 구조를 제공하며, 현재 구현에서는 관리자 인증, 식당 정보 관리, 리뷰 CSV 업로드, 분석 작업 생성 및 상태 조회 기능을 연동한다. 관리자 변경 이력 저장과 역할 기반 권한 분리는 향후 확장 단계에서 추가할 수 있다.

Page	Description
AdminLoginPage	관리자 인증을 위한 로그인 화면
AdminDashboardPage	등록된 식당 수, 리뷰 수, 분석 작업 현황 등을 요약 표시
AdminRestaurantsPage	식당 정보 등록, 수정, 삭제 관리
AdminReviewsPage	식당별 리뷰 데이터 업로드 및 조회
AdminAnalysesPage	LLM 분석 작업 실행, 진행률 확인, 분석 결과 확인

4.5 Component Design

프론트엔드는 재사용 가능한 컴포넌트를 기반으로 구성된다. 각 컴포넌트는 특정 화면 요소 또는 기능을 담당하며, 페이지 컴포넌트는 이러한 하위 컴포넌트들을 조합하여 하나의 화면을 구성한다.

Component	Responsibility
MainLayout	일반 사용자 화면의 공통 레이아웃 제공
AdminLayout	관리자 화면의 사이드바, 헤더, 인증 상태 확인, 관리자 라우트 보호
KakaoMap	Kakao Map API 기반 지도 렌더링 및 식당 마커 표시
FilterPanel	음식 종류, 가격대, 주차, 웨이팅, 정렬 기준 입력
RestaurantCard	개별 식당의 요약 정보, 태그, 추천 점수, 추천 근거 표시
RestaurantList	식당 목록 렌더링
TagBadge	식당 태그를 시각적으로 표시
ReasonChips	추천 근거를 짧은 문장 또는 칩 형태로 표시
Loading / Skeleton	데이터 로딩 상태 표시
ErrorMessage	API 오류 또는 사용자 안내 메시지 표시
EmptyState	조건에 맞는 식당이 없을 때 빈 결과 상태 표시
ErrorBoundary	지도 또는 특정 UI 영역 오류 발생 시 fallback UI 제공

향후 추가 또는 보완이 필요한 컴포넌트는 다음과 같다.

Planned Component	Purpose
ReviewUploadForm	관리자용 리뷰 또는 LLM/Data 결과 파일 업로드
AnalysisJobStatus	분석 작업 진행률 및 상태 표시

SummaryEditForm	관리자용 식당 요약 및 태그 수정
ChangeLogViewer	관리자 데이터 수정 이력 확인
FilterOptionManager	필터 항목을 관리자 화면에서 관리

4.6 Frontend Data Flow

프론트엔드의 데이터 흐름은 일반 사용자 탐색 흐름과 관리자 데이터 관리 흐름으로 구분된다.

4.6.1 General User Data Flow

일반 사용자 식당 탐색 흐름은 다음과 같다.

1. 사용자가 / 경로에서 캠퍼스를 선택한다.
2. 프론트엔드는 선택된 캠퍼스의 slug를 기반으로 /campus/:slug로 이동한다.
3. HomePage는 campus_slug를 기준으로 백엔드 API에 식당 목록을 요청한다.
4. 필터가 적용되지 않은 경우 캠퍼스별 식당 목록 API를 호출하고, 필터가 적용된 경우 추천 API를 호출한다.
5. 백엔드는 식당 기본 정보, 식당 요약, 태그, 평균 리뷰 신뢰도 점수, 추천 점수, 추천 근거를 반환한다.
6. 프론트엔드는 반환된 데이터를 지도 마커와 리스트 카드에 동시에 표시한다.
7. 사용자가 특정 식당을 선택하면 /restaurants/:id로 이동하여 상세 정보를 조회한다.

4.6.2 Admin Data Flow

관리자 데이터 관리 흐름은 다음과 같다.

1. 관리자가 /admin/login에서 로그인한다.
2. 인증에 성공하면 관리자 대시보드로 이동한다.
3. 관리자는 식당 기본 정보, 식당별 요약 정보, 식당 태그 또는 LLM/Data 결과 파일을 등록하거나 수정할 수 있다.
4. 백엔드는 업로드 또는 import된 데이터를 검증한 뒤 데이터베이스에 저장한다.
5. 관리자는 저장된 요약과 태그를 확인하고, 필요한 경우 수정한다.
6. 수정된 결과는 일반 사용자 화면의 식당 리스트, 상세 정보, 추천 근거에 반영된다.

리뷰 CSV 업로드와 분석 작업 생성 및 상태 조회는 현재 관리자 API 범위에 포함된다. 관리자가 리뷰 CSV 파일을 업로드하면 백엔드는 restaurant_external_id를 기준으로 식당을 매칭하고, 리뷰 원문을 reviews 테이블에 저장한다. 또한 관리자는 특정 식당에 대한 분석 작업을 생성하고, 프론트엔드는 pending 또는 running 상태의 작업을 일정 주기로 polling하여 진행 상태를 확인할 수 있다. 다만 실제 LLM 모델을 호출하여 리뷰별 세부 분석 결과를 review_analyses 테이블에 저장하는 기능과 분석 실패 재시도 로직은 향후 확장 단계에서 구현한다.

4.7 Kakao Map Integration Design

프론트엔드는 지도 기반 식당 탐색을 위해 Kakao Map JavaScript API를 사용한다. 지도는 사용자의 실시간 GPS 위치가 아니라, 사용자가 선택한 campus_slug에 대응되는 캠퍼스 중심 좌표를 기준으로 렌더링된다. 식당 위치는 백엔드 API 응답에 포함된 lat, lng 값을 이용하여 마커로 표시한다.

지도 연동은 KakaoMap 컴포넌트와 useKakaoLoader hook을 중심으로 구성된다. useKakaoLoader는 Kakao Map SDK를 필요한 시점에 로드하고, KakaoMap 컴포넌트는 캠퍼스 중심 좌표와 식당 좌표 데이터를 바탕으로 지도와 마커를 표시한다. 사용자가 마커를 클릭하면 식당 상세 페이지로 이동하거나 식당의 간략 정보를 확인할 수 있다.

Kakao Map API 로딩 실패, API Key 누락, 네트워크 오류 등이 발생하더라도 사용자가 식당 정보를 계속 확인할 수 있도록 리스트 기반 fallback 화면을 제공한다. 또한 Kakao Map API Key는 코드에 직접 작성하지 않고 환경 변수 또는 별도 설정 파일을 통해 관리한다.

4.8 State Management Design

프론트엔드는 상태의 성격에 따라 URL 상태, 서버 상태, 전역 UI 상태, 로컬 컴포넌트 상태를 구분하여 관리한다. 이를 통해 필터 조건, API 응답 데이터, 관리자 인증 상태, 지도/리스트 화면 상태가 서로 불필요하게 결합되지 않도록 한다.

프론트엔드에서 관리해야 하는 주요 상태는 다음과 같다.

State Type	Tool	Managed Data
URL State	React Router, useFilterParams	campus_slug, category, priceRange, parking, sort 등 필터 조건
Server State	TanStack Query	캠퍼스별 식당 목록, 필터 기반 추천 결과, 식당 상세 정보 API 응답
Global UI/Auth State	Zustand	관리자 인증 토큰, 관리자 로그인 상태, 전역 UI 상태
Local Component State	useState, useEffect	지도/리스트 보기 모드, 선택된 식당 ID, 일시적 로딩 상태, UI 토글 상태

필터 조건은 URL query parameter와 동기화하여 사용자가 새로고침하거나 링크를 공유해도 동일한 탐색 조건을 유지할 수 있도록 한다. 식당 목록과 추천 결과는 TanStack Query를 통해 캐싱하고, 동일 요청의 불필요한 재호출을 줄인다. 관리자 인증 상태는 Zustand store에서 관리하며, 인증되지 않은 사용자는 관리자 라우트에 접근할 수 없도록 한다.

4.9 Error and Fallback Design

프론트엔드는 오류 상황에서도 사용자가 시스템을 계속 이용할 수 있도록 오류 안내와 fallback UI를 제공해야 한다. 요구사항 명세서에서도 데이터 없음, 지도 로드 실패, 분석 실패 등의 상황에 적절한 메시지를 제공해야 한다고 정의하고 있다.

주요 오류 상황과 프론트엔드 대응 방식은 다음과 같다.

Error Situation	Frontend Response
Kakao Map API 로드 실패	지도 오류 메시지를 표시하고 /campus/:slug/list 또는 리스트 모드로 이동할 수 있게 한다.
캠퍼스 정보 조회 실패	"캠퍼스 정보를 불러올 수 없습니다." 메시지와 홈 이동 버튼을 제공한다.
식당 목록 조회 실패	오류 메시지와 재시도 버튼을 제공한다.
필터 조건에 맞는 식당 없음	"선택한 조건에 맞는 식당이 없습니다." 메시지와 필터 초기화 버튼을 제공한다.
식당 상세 정보 없음	기본 오류 메시지와 이전 화면으로 돌아가기 버튼을 제공한다.
분석 정보 없음	AI 요약 영역에 "분석 정보가 아직 충분하지 않습니다." 안내를 표시한다.
관리자 인증 실패	로그인 실패 메시지를 표시하고 관리자 로그인 화면에 머무르게 한다.
관리자 데이터 업로드 실패	실패 원인과 재업로드 안내를 표시한다.
분석 작업 실패	관리자 분석 화면에 실패 상태와 오류 메시지를 표시한다.

오류 메시지는 기술적인 내부 오류를 그대로 노출하지 않고, 사용자가 다음 행동을 이해할 수 있는 형태로 제공해야 한다. 예를 들어 "500 Internal Server Error"를 그대로 표시하는 대신 "식당 정보를 불러오는 중 문제가 발생했습니다. 잠시 후 다시 시도해주세요."와 같이 안내한다. 또한 지도 API 실패와 같이 외부 서비스 의존성에서 발생한 오류는 전체 서비스 중단으로 처리하지 않고, 리스트 기반 탐색으로 전환하여 핵심 기능을 유지한다.

4.10 Frontend Sequence Diagrams

본 절에서는 프론트엔드 관점에서 주요 사용자 흐름을 시퀀스 형태로 설명한다. 상세 API 요청 및 응답 형식은 6장 Protocol Design에서 다루므로, 본 절에서는 화면 전환과 데이터 요청 흐름에 초점을 둔다.

4.10.1 Campus Selection and Restaurant List Loading

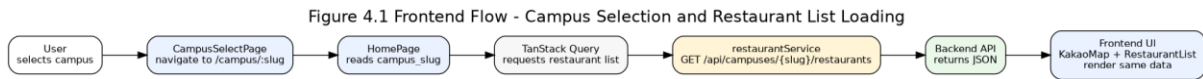


Figure 4.1 Campus Selection and Restaurant List Loading

사용자가 CampusSelectPage에서 자연과학캠퍼스 또는 인문사회캠퍼스를 선택하면 프론트엔드는 선택된 campus_slug를 기준으로 /campus/:slug 경로로 이동한다. HomePage는 URL에서 campus_slug를 읽고, TanStack Query를 통해 캠퍼스별 식당 목록 API를 호출한다. 백엔드 API는 해당 캠퍼스에 속한 식당 정보, 식당 요약, 태그, 평균 리뷰 신뢰도 점수를 JSON으로 반환한다. 프론트엔드는 동일한 응답 데이터를 KakaoMap과 RestaurantList에 전달하여 지도 마커와 리스트 카드가 항상 같은 식당 데이터를 기반으로 표시되도록 한다.

4.10.2 Filter-based Recommendation Update



Figure 4.2 Filter-based Recommendation Update

사용자가 FilterPanel에서 음식 종류, 가격대, 주차 가능 여부, 웨이팅 여부 등의 조건을 변경하면 선택된 필터 상태는 useFilterParams를 통해 URL query parameter와 동기화된다. 필터 조건이 변경되면 TanStack Query는 변경된 query key를 기준으로 추천 API를 다시 호출한다. 백엔드는 필터 조건과 식당의 카테고리, 가격대, 태그, 평균 리뷰 신뢰도 점수를 바탕으로 추천 점수를 계산하고, recommendationScore와 recommendationReasons를 포함한 JSON 응답을 반환한다. 프론트엔드는 이 결과를 바탕으로 리스트 정렬 순서와 지도 마커 강조 상태를 갱신한다.

4.10.3 Kakao Map Failure and List Fallback

Figure 4.3 Frontend Flow - Kakao Map Failure and List Fallback

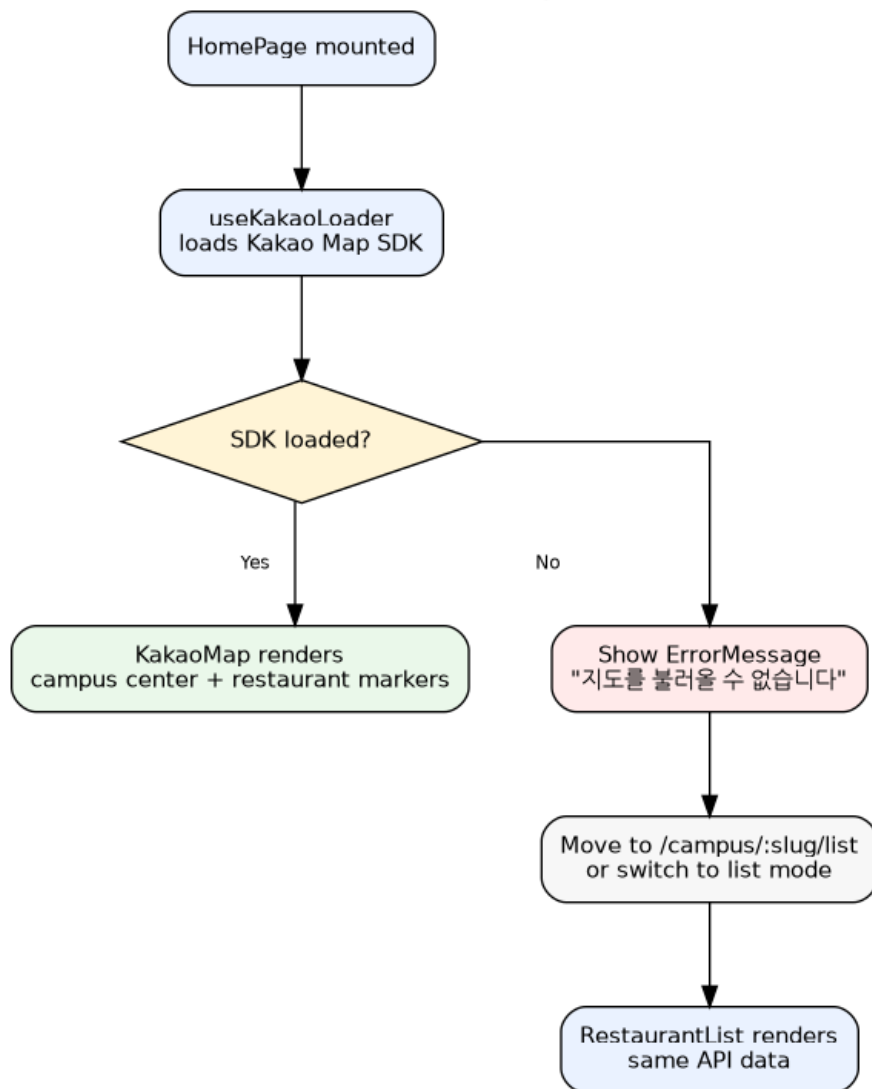


Figure 4.3 Kakao Map Failure and List Fallback

HomePage가 렌더링되면 useKakaoLoader는 Kakao Map SDK를 로드한다. SDK 로딩에 성공하면 KakaoMap 컴포넌트는 선택된 캠퍼스 중심 좌표와 식당 좌표를 바탕으로 지도와 마커를 표시한다. 반면 Kakao Map API 로딩 실패, API Key 누락, 네트워크 오류가 발생하면 프론트엔드는 지도 오류 메시지를 표시하고, 사용자가 리스트 기반 탐색을 계속할 수 있도록 /campus/:slug/list 또는 리스트 모드로 전환한다. 이때 리스트 화면은 지도 화면과 동일한 백엔드 API 데이터를 사용하므로, 지도 API 실패 상황에서도 식당 탐색이라는 핵심 기능은 유지된다.

4.11 Frontend Design Constraints

프론트엔드 설계는 다음과 같은 제약사항을 따른다.

첫째, 일반 사용자는 로그인 없이 서비스를 이용할 수 있어야 한다. 따라서 일반 사용자 화면에서는 개인정보 입력을 요구하지 않는다.

둘째, MVP 단계에서는 사용자의 실시간 GPS 위치를 사용하지 않는다. 사용자는 자연과학캠퍼스 또는 인문사회캠퍼스 중 하나를 직접 선택하며, 시스템은 선택된 캠퍼스 중심 좌표를 기준으로 식당 정보를 제공한다.

셋째, 지도 기반 탐색은 Kakao Map API에 의존하므로, 지도 API 실패 상황에 대비한 리스트 기반 fallback 화면을 제공해야 한다.

넷째, LLM 분석 결과는 사용자가 이해하기 쉬운 형태로 표현해야 한다. "가짜 리뷰"와 같은 단정적인 표현은 피하고, "광고성 가능성", "신뢰도 점수", "리뷰 기반 요약"과 같은 참고 정보 형태로 제공한다.

다섯째, 관리자 화면은 일반 사용자 화면과 분리되어야 하며, 최종 구현에서는 인증된 관리자만 접근할 수 있어야 한다.

여섯째, 모바일 환경에서도 캠퍼스 선택, 리스트 조회, 식당 상세 정보 확인은 가능해야 한다. 다만 지도와 리스트를 동시에 표시하는 화면은 데스크톱 또는 태블릿 환경을 우선 지원한다.

일곱째, API Key, 서버 URL 등 환경별 설정값은 코드 내부에 직접 작성하지 않고 환경 변수 또는 설정 파일을 통해 관리한다.

4.12 Frontend Future Improvements

현재 프론트엔드는 주요 화면 구조와 라우팅을 중심으로 구현된 상태이며, 향후 다음 항목을 보완해야 한다.

첫째, 모바일 화면에서 지도와 리스트 표시 방식을 반응형으로 개선해야 한다.

둘째, 오류 메시지와 fallback UI를 공통 컴포넌트로 정리하여 화면 간 일관성을 유지해야 한다.

System Architecture - Backend

5.1 Objectives

본 장에서는 LLM 기반 리뷰 신뢰도 분석 맛집 추천 시스템의 백엔드 아키텍처를 설명한다. 백엔드는 프론트엔드 웹 클라이언트, 데이터베이스, LLM/Data 결과 사이의 중간 계층으로 동작하며, 캠퍼스 및 식당 정보 조회, 필터 기반 추천 점수 계산, 식당 상세 정보 제공, LLM/Data 결과 import, 관리자 데이터 관리 기능을 담당한다.

본 시스템의 백엔드는 FastAPI와 SQLAlchemy를 기반으로 구성된다. MVP 단계에서는 캠퍼스, 식당, 식당 요약, 식당 태그 데이터를 중심으로 일반 사용자 기능을 제공한다. 또한 프론트엔드 관리자 화면과의 연동을 위해 관리자 인증, 식당 CRUD, 리뷰 CSV 업로드, 분석 작업 생성 및 상태 조회 기능을 함께 제공한다. 이를 위해 현재 데이터베이스에는 reviews와 analysis_jobs 테이블을 포함한다. 다만 리뷰별 세부 분석 결과 관리, 다중 관리자 계정, 관리자 변경 이력 관리는 향후 확장 가능한 구조로 분리한다.

본 장의 목적은 백엔드의 주요 모듈, 모듈 간 책임 분리, 사용자 요청 흐름, LLM/Data 결과 반영 흐름, 백엔드 설계 제약사항을 정의하는 것이다. 세부 API 요청 및 응답 형식은 6장 Protocol Design에서 다루고, 데이터베이스 테이블 구조는 7장 Database Design에서 상세히 설명한다.

5.2 Backend Overall Structure

Backend API Server는 프론트엔드 요청을 처리하고, 데이터베이스에 저장된 식당 정보와 LLM/Data 분석 결과를 조합하여 JSON 형태로 반환한다. 백엔드는 사용자 요청 시점에 LLM을 실시간으로 호출하지 않고, 사전에 저장된 식당 요약, 태그, 평균 리뷰 신뢰도 점수를 조회하여 응답을 구성한다.

백엔드는 일반 사용자용 식당 탐색 API, 추천 계산 로직, 관리자 데이터 관리 기능, LLM/Data 결과 반영 기능으로 구성된다. 일반 사용자용 기능은 캠퍼스 목록 조회, 캠퍼스별 식당 목록 조회, 필터 기반 추천, 식당 상세 조회를 담당한다. 관리자 기능은 단일 관리자 인증, 식당 정보 관리, 리뷰 CSV 업로드, 분석 작업 생성 및 상태 조회를 담당한다.

Backend Module	Responsibility
User Query API	캠퍼스 목록, 캠퍼스별 식당 목록, 식당 상세 정보를 조회하여 Frontend Web Client에 제공한다.
Recommendation Service	필터 조건, 식당 속성, 태그, Average Trust Score를 바탕으로 Recommendation Score와 Recommendation Reasons를 계산한다.
Admin API	단일 관리자 인증, 식당 정보 관리, 리뷰 CSV 업로드, 분석 작업 생성 및 상태 조회를 처리한다.
LLM/Data Import Service	LLM/Data 결과 파일을 검증하고 식당 기본 정보, 식당 요약, 식당 태그 데이터를 Database에 저장한다.
Database and Error Handling Layer	SQLAlchemy 기반 데이터베이스 접근 로직을 분리하고, 데이터 없음, 잘못된 요청, DB 오류에 대한 일관된 오류 응답을 제공한다.

현재 구현에서는 리뷰 원문 저장과 분석 작업 상태 관리까지 백엔드 범위에 포함된다. 관리자는 인증 후 식당 정보를 관리하고, 리뷰 CSV 파일을 업로드하며, 특정 식당에 대한 분석 작업을 생성하고 상태를 조회할 수 있다. 다만 리뷰별 세부 신뢰도 분석 결과를 별도 테이블에 저장하는 기능, 다중 관리자 계정 관리, 관리자 변경 이력 관리는 현재 구현 범위에 포함하지 않고 향후 확장 항목으로 분리한다.

각 모듈의 세부 동작은 5.3 Backend Components에서 설명하고, 구체적인 API 요청 및 응답 형식은 6장 Protocol Design에서 다룬다.

5.3 Backend Components

5.3.1 Campus and Restaurant Query Module

Campus and Restaurant Query Module은 캠퍼스 목록과 캠퍼스별 식당 목록을 조회하는 기능을 담당한다. 프론트엔드는 사용자가 선택한 `campus_slug`를 기준으로 백엔드에 식당 목록을 요청하고, 백엔드는 해당 slug에 대응되는 캠퍼스를 조회한 뒤 해당 캠퍼스에 속한 식당 목록을 반환한다.

캠퍼스 식별에는 내부 DB ID보다 `campus_slug`를 우선 사용한다. 이는 프론트엔드 라우팅이 `/campus/:slug` 구조를 사용하기 때문에, 라우팅 값과 API 요청 값의 일관성을 유지하기 위한 설계이다. 백엔드는 내부적으로 `campus_id`를 사용하여 `restaurants` 테이블과 연결하지만, API 경계에서는 `natural`, `humanities`와 같은 slug 값을 사용한다.

식당 목록 응답은 식당 기본 정보, 위치 정보, 요약, 태그, 평균 리뷰 신뢰도 점수를 포함한다. DB 내부 컬럼명은 `snake_case`를 사용하고, 프론트엔드 응답 필드는 `camelCase` 형식으로 변환한다.

5.3.2 Restaurant Detail Module

Restaurant Detail Module은 사용자가 특정 식당을 선택했을 때 상세 정보를 제공하는 기능을 담당한다. 프론트엔드는 식당 ID를 기준으로 상세 정보 API를 호출하고, 백엔드는 `restaurants`, `restaurant_summaries`, `restaurant_tags` 데이터를 조합하여 식당 상세 응답을 생성한다.

상세 정보에는 식당명, 주소, 위치 좌표, 음식 종류, 가격대, 원본 링크, 대표 메뉴, 식당 요약, 분위기 요약, 주차 요약, 웨이팅 요약, 평균 리뷰 신뢰도 점수, 태그 정보가 포함된다. 분석 결과가 아직 없는 식당의 경우 `hasAnalysis` 값을 `false`로 반환하거나, 요약 및 태그 필드를 비워 두어 프론트엔드가 대체 안내 메시지를 표시할 수 있도록 한다.

이 모듈은 사용자가 원본 리뷰 전체를 읽지 않아도 식당의 핵심 특징을 빠르게 파악할 수 있도록, 데이터베이스에 저장된 요약과 태그 정보를 사용자 친화적인 응답 형식으로 변환하는 역할을 수행한다.

5.3.3 Recommendation Service

Recommendation Service는 사용자가 선택한 필터 조건을 바탕으로 식당별 추천 점수를 계산하고, 추천 근거를 생성하는 모듈이다. MVP 단계에서는 복잡한 개인화 모델을

사용하지 않고, 식당의 기본 속성, 식당 태그, 평균 리뷰 신뢰도 점수를 활용한 rule-based scoring 방식을 사용한다.

사용자가 음식 종류, 가격대, 분위기, 주차 가능 여부와 같은 필터를 선택하면, 백엔드는 해당 조건과 식당의 category, price_level, restaurant_tags, average_trust_score를 비교한다. 조건이 일치하는 경우 항목별 가중치를 더하고, 평균 리뷰 신뢰도 점수를 일부 반영하여 최종 recommendationScore를 계산한다.

MVP 추천 점수 계산 기준은 다음과 같다.

Condition	Score
category가 선택한 음식 종류와 일치	+25
price_level이 선택한 가격대와 일치	+20
mood 태그가 선택한 분위기와 일치	+20
parking 태그가 선택한 주차 조건과 일치	+15
평균 리뷰 신뢰도 점수 반영	$\text{average_trust_score} \times 0.2$

추천 API는 점수만 반환하지 않고 recommendationReasons를 함께 반환한다. 예를 들어 음식 종류가 일치한 경우 “음식 종류가 선택한 조건과 일치합니다.”, 분위기 태그가 일치한 경우 “데이트 분위기 관련 태그가 있습니다.”와 같은 설명을 제공한다. 이를 통해 사용자는 추천 순위가 어떤 조건에 의해 결정되었는지 이해할 수 있다.

향후에는 리뷰 수, 낮은 신뢰도 리뷰 비율, 높은 신뢰도 리뷰 비율, 메타데이터 완성도, 사용자 피드백 등을 추가로 반영하여 추천 로직을 고도화할 수 있다. 그러나 MVP 단계에서는 설명 가능성과 구현 안정성을 우선하여 규칙 기반 추천 방식을 사용한다.

5.3.4 LLM/Data Import Service

LLM/Data Import Service는 LLM/Data 팀이 생성한 CSV 또는 JSON 결과를 Backend API Server의 Database에 반영하는 모듈이다. 이 모듈은 사용자 요청 시점에 LLM을 실시간으로 호출하는 방식이 아니라, 사전에 생성된 LLM/Data 결과를 검증하고 저장한 뒤 일반 사용자 API 응답에 활용하는 방식으로 동작한다.

백엔드가 import하는 주요 CSV는 다음과 같다.

CSV File	Purpose	Key Link
restaurants.csv	식당 기본 정보 저장	external_id
restaurant_summaries.csv	식당별 요약 정보와 Average Trust Score 저장	restaurant_external_id → restaurants.external_id
restaurant_tags.csv	식당별 태그 정보 저장	restaurant_external_id → restaurants.external_id

백엔드는 import 과정에서 campus_slug가 유효한지, external_id가 중복되지 않는지, 필수 좌표와 식당명이 존재하는지 확인한다. 또한 restaurant_summaries.csv와 restaurant_tags.csv의 restaurant_external_id가 restaurants.csv 또는 기존 Database의 restaurants.external_id와 연결되는지 검증한다.

external_id는 LLM/Data 결과와 백엔드 식당 데이터를 연결하는 핵심 필드이다. 백엔드 내부에서는 restaurants.id를 식당의 내부 식별자로 사용하지만, 외부 데이터 또는 LLM/Data 결과와 연결할 때는 restaurants.external_id를 기준으로 사용한다. 이 구조를 통해 LLM/Data 파트와 백엔드 파트가 독립적으로 작업하면서도 동일한 식당 데이터를 안정적으로 연결할 수 있다.

5.3.5 Review Credibility Score Handling

Review Credibility Score Handling은 LLM/Data 분석 결과로 생성된 리뷰 신뢰도 점수를 백엔드 응답과 추천 계산에 반영하는 방식이다. LLM/Data Analysis Module은 skt/A.X-4.0-Light 기반 LoRA 파인튜닝 모델을 사용하여 리뷰 텍스트가 비협찬성 리뷰일 가능성을 0에서 1 사이의 확률값으로 산출하고, 이를 100점 기준의 리뷰 신뢰도 점수로 환산한다.

$$\text{ReviewCredibilityScore} = \text{non_sponsored_probability} \times 100$$

이 점수는 리뷰의 실제 진위 여부를 단정하는 값이 아니라, 리뷰 이벤트 또는 협찬성 표현의 영향을 상대적으로 적게 받았을 가능성을 나타내는 참고 지표이다. 따라서 백엔드는 이 값을 “가짜 리뷰 판별 결과”로 제공하지 않고, averageTrustScore 또는 식당 요약 정보의 일부로 사용자 화면에 전달한다.

MVP 단계에서는 식당 단위의 average_trust_score를 restaurant_summaries 테이블에 저장하고, 추천 계산에서는 average_trust_score를 일부 가중치로 반영한다. 향후 확장 단계에서는 리뷰별 점수, 낮은 신뢰도 리뷰 비율, 높은 신뢰도 리뷰 비율, 모델 버전, 분석 시각 등을 별도 테이블에 저장하여 더 정교한 식당 단위 신뢰도 지표를 계산할 수 있다.

5.3.6 Admin Data Management Module

Admin Data Management Module은 데이터 관리자가 식당 정보와 LLM/Data import 결과를 관리할 수 있도록 지원하는 모듈이다. 현재 구현에서는 단일 관리자 인증, 식당 기본 정보 관리, 리뷰 CSV 업로드, 분석 작업 생성 및 상태 조회를 제공한다.

식당별 요약, 대표 메뉴, 분위기, 주차, 웨이팅 요약, 태그 값의 세부 수정 기능은 향후 관리자 기능 고도화 단계에서 확장할 수 있다. 또한 다중 관리자 계정, 역할 기반 권한 분리, 관리자 변경 이력 저장은 현재 구현 범위에 포함하지 않고 향후 확장 항목으로 분리한다.

5.3.7 Admin Authentication Module

Admin Authentication Module은 관리자 화면 접근을 위한 로그인과 인증 처리를 담당한다. 관리자는 `/api/admin/auth/login` 엔드포인트에 username과 password를 전달하고, 백엔드는 환경 변수에 저장된 관리자 계정 정보와 비교한다. 인증에 성공하면 백엔드는 JWT access token을 발급하며, 이후 관리자 API 요청은 Bearer token을 통해 인증된다.

현재 구현에서는 관리자 계정을 별도의 데이터베이스 테이블에 저장하지 않는다. 따라서 다중 관리자 계정, 역할 기반 권한 분리, 관리자 비밀번호 변경 기능은 향후 확장 항목으로 둔다.

5.3.8 Admin Restaurant Management Module

Admin Restaurant Management Module은 관리자가 식당 데이터를 조회, 등록, 수정, 삭제할 수 있도록 지원한다. 식당 등록 시 백엔드는 campusSlug를 기준으로 캠퍼스를 조회하고, 식당의 이름, 주소, 좌표, 카테고리, 가격대, 전화번호, 원본 URL, externalId를 저장한다. externalId가 전달되지 않은 경우 백엔드는 관리자 등록 식당임을 구분할 수 있는 임시 externalId를 생성한다.

식당 삭제는 실제 레코드를 제거하는 방식이 아니라 `is\_active` 값을 false로 변경하는 soft delete 방식으로 처리한다. 이를 통해 기존 리뷰, 요약, 태그, 분석 작업과의 관계를 보존하면서 사용자 화면에서는 비활성화된 식당을 제외할 수 있다.

5.4 Backend Data Flow

5.4.1 User Request Flow

일반 사용자 요청 흐름은 캠퍼스별 식당 목록 조회, 필터 기반 추천, 식당 상세 조회로 구분된다. 프론트엔드가 campus_slug 또는 restaurant_id를 전달하면, 백엔드는 해당 값에 맞는 캠퍼스와 식당 데이터를 조회한다. 이후 식당 기본 정보, 요약, 태그, Average Trust Score를 조합하여 사용자 화면에 필요한 응답을 구성한다.

필터 조건이 포함된 요청의 경우, 백엔드는 식당의 category, price_level, restaurant_tags, average_trust_score를 기준으로 Recommendation Score와 Recommendation Reasons를 계산한다. 프론트엔드는 이 결과를 식당 리스트 정렬, 지도 마커 표시, 식당 상세 정보 화면에 반영한다. 구체적인 endpoint와 request/response 형식은 6장 Protocol Design에서 설명한다.

5.4.2 LLM/Data Import Flow

LLM/Data 결과 반영 흐름은 사용자 요청 흐름과 분리된다. LLM/Data 담당자는 식당 기본 정보, 식당별 요약, 식당별 태그, 평균 리뷰 신뢰도 점수를 CSV 또는 JSON 형태로 준비한다. 백엔드는 import 과정을 통해 해당 데이터를 검증하고 데이터베이스에 저장한다.

기본 import 흐름은 다음과 같다.

LLM/Data CSV 또는 JSON

→ import validation

→ restaurants 저장 또는 갱신

→ restaurant_summaries 저장 또는 갱신

→ restaurant_tags 저장 또는 갱신

→ FastAPI REST API를 통한 사용자 화면 제공

MVP 단계에서는 restaurants.csv, restaurant_summaries.csv, restaurant_tags.csv를 통해 식당 기본 정보, 식당 요약, 식당 태그를 DB에 저장한다. 현재 구현에서는 관리자 리뷰 업로드와 분석 작업 상태 관리를 위해 reviews와 analysis_jobs를 함께 사용한다. 향후 확장 단계에서는 리뷰별 세부 분석 결과와 식당 단위 분석 통계를 저장하기 위해 review_analyses와 restaurant_analyses를 추가할 수 있다.

5.5 Backend Design Constraints

백엔드 설계는 다음 제약사항을 따른다.

첫째, MVP 단계에서는 사용자 요청마다 실시간 LLM 분석을 수행하지 않는다. LLM/Data 분석 결과는 사전에 생성되어 DB에 저장되고, 사용자 화면에서는 저장된 결과만 조회한다.

둘째, 프론트엔드 라우팅과 API 연결을 위해 campus_slug를 사용한다. 백엔드는 내부적으로 campus_id를 사용할 수 있지만, API 경계에서는 /api/campuses/{slug}/restaurants와 같은 slug 기반 엔드포인트를 제공한다.

셋째, LLM/Data 결과와 백엔드 DB 식당 데이터를 연결하기 위해 Restaurant.external_id를 사용한다. external_id는 외부 데이터 기준 식당 식별자이며, 백엔드 내부 ID와 구분된다.

넷째, 지도 마커 표시를 위해 식당의 위도와 경도를 저장하고, API 응답에서는 lat, lng 형식으로 반환한다.

다섯째, 추천 결과는 점수뿐 아니라 추천 근거를 함께 제공해야 한다.

recommendationScore만 제공하면 사용자가 추천 결과를 이해하기 어렵기 때문에, recommendationReasons를 함께 반환한다.

여섯째, DB 내부 필드명과 프론트엔드 응답 필드명을 분리한다. DB에서는 snake_case를 사용하고, API 응답에서는 프론트엔드에서 사용하기 쉬운 camelCase 형식을 사용한다.

일곱째, 리뷰 신뢰도 점수는 참고 지표로만 제공한다. 백엔드는 리뷰 또는 식당을 가짜, 조작, 광고 식당으로 단정하는 표현을 API 응답이나 사용자 화면에 직접 제공하지 않는다.

5.6 Backend Future Improvements

향후 백엔드는 PostgreSQL 전환, 리뷰별 분석 결과 저장, 식당 단위 분석 통계 분리, 다중 관리자 계정 및 변경 이력 관리, 분석 작업 재시도 로직, 모델 버전 관리 기능을 확장할 수 있다. 자세한 향후 확장 항목은 10.5 Future Extension Items에서 정리한다.

Protocol Design

6.1 Objectives

본 장에서는 Frontend Web Client와 Backend API Server 사이의 통신 방식을 정의한다. 본 시스템은 React 기반 프론트엔드와 FastAPI 기반 백엔드가 HTTP/HTTPS 기반 REST API와 JSON 형식으로 데이터를 주고받는 구조를 사용한다.

MVP 단계에서 백엔드는 campuses, restaurants, restaurant_summaries, restaurant_tags를 기반으로 캠퍼스 목록 조회, 캠퍼스별 식당 목록 조회, 식당 상세 조회, 필터 기반 추천 API를 제공한다. 프론트엔드는 이 API 응답을 지도 마커, 리스트 카드, 식당 상세 페이지, 추천 근거 표시 영역에 반영한다.

본 장의 목표는 다음과 같다. 첫째, 프론트엔드와 백엔드 사이의 기본 통신 형식을 정의한다. 둘째, MVP 단계에서 사용하는 API endpoint와 요청 파라미터를 정리한다. 셋째, DB 내부 필드명과 프론트엔드 응답 필드명의 변환 규칙을 정의한다. 넷째, 추천 결과와 오류 응답의 JSON 형식을 설명한다. 다섯째, 일반 사용자 API와 관리자 API의 범위를 구분하고, 현재 구현된 관리자 연동 기능과 향후 확장 API의 방향을 제시한다.

6.2 Communication Protocol

프론트엔드와 백엔드는 HTTP/HTTPS 기반 REST API를 통해 통신한다. 개발 환경에서는 HTTP를 사용할 수 있으나, 운영 환경에서는 관리자 인증 정보, API 요청 데이터, 데이터 관리 요청을 보호하기 위해 HTTPS 적용을 전제로 한다.

모든 API 요청과 응답의 기본 데이터 형식은 JSON이다. 일반 사용자 기능에서는 캠퍼스 목록 조회, 캠퍼스별 식당 목록 조회, 필터 기반 추천, 식당 상세 조회 요청이 발생한다. 현재 구현된 관리자 기능에서는 관리자 인증, 리뷰 CSV 업로드, 분석 작업 생성 및 상태 조회

요청이 발생한다. 식당 요약 및 태그 수정, 리뷰별 세부 분석 결과 조회, 관리자 변경 이력 조회는 향후 확장 단계에서 추가할 수 있다.

백엔드는 요청 처리 결과에 따라 적절한 HTTP 상태 코드를 반환한다. 정상 조회는 200 OK를 사용하고, 잘못된 요청 파라미터는 400 Bad Request 또는 422 Unprocessable Entity를 사용한다. 존재하지 않는 캠퍼스나 식당은 404 Not Found를 사용하며, 관리자 인증 실패는 401 Unauthorized, 권한 부족은 403 Forbidden으로 처리한다.

6.3 API Design Principles

본 시스템의 API 설계는 다음 원칙을 따른다.

첫째, API endpoint는 리소스 중심으로 설계한다. 캠퍼스, 식당, 추천 결과, 관리자 데이터와 같은 리소스를 기준으로 URL을 구성한다.

둘째, 프론트엔드 라우팅과 API 식별자는 campus_slug를 기준으로 연결한다. 프론트엔드는 /campus/:slug 경로를 사용하므로, 백엔드도 /api/campuses/{slug}/restaurants와 같은 slug 기반 API를 제공한다.

셋째, DB 내부 필드명과 API 응답 필드명을 분리한다. 데이터베이스와 SQLAlchemy 모델에서는 snake_case를 사용하고, 프론트엔드 응답에서는 camelCase를 사용한다.

넷째, 사용자 요청 시점에는 LLM을 실시간으로 호출하지 않는다. 사용자 API는 사전에 저장된 식당 요약, 태그, 평균 리뷰 신뢰도 점수를 조회하여 응답한다.

다섯째, 추천 API는 점수뿐 아니라 추천 근거를 함께 제공한다. recommendationScore와 recommendationReasons를 함께 반환하여 추천 결과의 설명 가능성을 확보한다.

여섯째, 리뷰 신뢰도 점수는 참고 지표로만 제공한다. API 응답과 사용자 화면에서는 리뷰의 진위 여부를 단정하는 표현을 사용하지 않는다.

6.4 Current MVP API Endpoints

현재 MVP 단계에서 사용되는 주요 API는 다음과 같다.

Method	Endpoint	Description	Used By
GET	/api/campuses	캠퍼스 목록 조회	CampusSelectPage
GET	/api/campuses/{slug}/restaurants	선택한 캠퍼스의 식당 목록 조회	HomePage, ListOnlyPage
GET	/api/campuses/{slug}/recommendations	필터 기반 추천 결과 조회	HomePage, ListOnlyPage
GET	/api/restaurants/{id}	식당 상세 정보 조회	RestaurantDetailPage

MVP 단계의 일반 사용자 화면은 위 네 가지 API를 중심으로 동작한다. 프론트엔드는 캠퍼스 선택 후 campus_slug를 기준으로 식당 목록 또는 추천 결과를 요청하고, 특정 식당을 선택하면 restaurant_id를 기준으로 상세 정보를 조회한다. 관리자 인증, 리뷰 업로드, 분석 작업 상태 조회 API는 일반 사용자 API와 분리하여 /api/admin prefix 아래에서 제공한다. 리뷰별 세부 분석 결과 조회, 신뢰 리뷰 후보 조회, 관리자 변경 이력 조회 API는 향후 확장 API로 분리한다.

6.5 Request Parameter Design

API 요청 파라미터는 path parameter와 query parameter로 구분한다. campus_slug와 restaurant_id처럼 특정 리소스를 식별하는 값은 path parameter로 전달하고, 필터 조건과 정렬 조건은 query parameter로 전달한다.

6.5.1 Main Request Parameters

주요 API 요청 파라미터는 다음과 같다. 캠퍼스와 식당처럼 특정 리소스를 식별하는 값은 path parameter로 전달하고, 음식 종류, 가격대, 분위기, 주차 여부와 같은 필터 조건은 query parameter로 전달한다.

API	Parameter Type	Name	Description
/api/campuses/{slug}/restaurants	Path	slug	캠퍼스 식별자. 예: natural, humanities
/api/campuses/{slug}/recommendations	Path	slug	추천 대상 캠퍼스 식별자

/api/campuses/{slug}/recommendations	Query	category	음식 종류 필터
/api/campuses/{slug}/recommendations	Query	price_level	가격대 필터
/api/campuses/{slug}/recommendations	Query	mood	분위기 태그 필터
/api/campuses/{slug}/recommendations	Query	parking	주차 가능 여부 필터
/api/restaurants/{id}	Path	id	식당 내부 ID

프론트엔드 내부 상태에서는 priceRange와 같은 camelCase 이름을 사용할 수 있지만, 백엔드 추천 API의 query parameter는 price_level과 같이 백엔드 필드명에 맞춘 이름을 사용할 수 있다. 이 경우 프론트엔드 service layer에서 priceRange를 price_level로 변환하여 요청한다. API 응답은 다시 프론트엔드 사용성을 고려하여 camelCase 형식으로 반환한다.

6.6 Restaurant List API

백엔드는 DB 내부 필드명을 그대로 노출하지 않고, 프론트엔드에서 사용하기 쉬운 응답 필드명으로 변환하여 JSON을 반환한다. 예를 들어 DB에서는 latitude, longitude, price_level, source_url, average_trust_score를 사용하지만, API 응답에서는 lat, lng, priceRange, sourceUrl, averageTrustScore를 사용한다.

6.6.1 Campus Restaurant List Request

이 API는 특정 캠퍼스 주변 식당 목록을 반환한다. 지도 마커 표시와 리스트 카드 렌더링에 모두 사용된다.

Item	Description
Method	GET
Endpoint	/api/campuses/{slug}/restaurants
Path Parameter	slug: natural 또는 humanities

Authentication	Not required
Request Body	None
Response	캠퍼스 정보와 식당 목록

식당 목록 응답에는 지도 표시를 위한 lat, lng와 리스트 표시를 위한 summary, tags, averageTrustScore, hasAnalysis가 함께 포함된다. 이를 통해 프론트엔드는 동일한 데이터를 지도 마커와 리스트 카드에 동시에 사용할 수 있다.

6.7 Recommendation API

필터 기반 추천 API는 사용자가 선택한 조건에 따라 식당 목록의 우선순위를 계산하고, 추천 점수와 추천 근거를 함께 반환한다. 프론트엔드는 FilterPanel에서 선택된 조건을 query parameter로 전달하고, 백엔드는 식당의 category, price_level, restaurant_tags, average_trust_score를 기반으로 recommendationScore를 계산한다.

Recommendation API는 다음 형식을 사용한다.

GET

/api/campuses/{slug}/recommendations?category={category}&price_level={price_level}&mood={mood}&parking={parking}

6.7.1 Recommendation Request

추천 API는 사용자가 선택한 필터 조건을 바탕으로 추천 점수를 계산하고, 추천 순서로 정렬된 식당 목록을 반환한다.

Item	Description
Method	GET
Endpoint	/api/campuses/{slug}/recommendations
Path Parameter	slug: 캠퍼스 식별자

Query Parameters	category, price_level, mood, parking
Authentication	Not required
Request Body	None
Response	추천 점수와 추천 근거가 포함된 식당 목록

Query Parameter

Parameter	Type	Required	Description
category	string	No	음식 종류
price_level	string	No	가격대
mood	string	No	분위기 태그
parking	boolean	No	주차 가능 여부

Request Example

GET

/api/campuses/natural/recommendations?category=양식&price_level=보통&mood=데이트&parking=true

6.7.2 Recommendation Response

추천 응답은 캠퍼스 정보와 추천 식당 목록을 포함한다. 각 식당은 recommendationScore와 recommendationReasons를 가진다.

```
{
  "campus": {
    "id": 1,
    "slug": "natural",
```

```

    "name": "성균관대학교 자연과학캠퍼스",
    "lat": 37.293,
    "lng": 126.974,
    "radiusM": 1500
  },
  "restaurants": [
    {
      "id": 1,
      "name": "예시 파스타집",
      "category": "양식",
      "lat": 37.294,
      "lng": 126.975,
      "priceRange": "보통",
      "summary": "데이트 분위기와 파스타 언급이 많은 식당입니다.",
      "averageTrustScore": 82.5,
      "recommendationScore": 81.5,
      "recommendationReasons": [
        "음식 종류가 선택한 조건과 일치합니다.",
        "데이트 분위기 관련 태그가 있습니다.",
        "리뷰 신뢰도 점수가 높은 편입니다."
      ],
      "hasAnalysis": true
    }
  ]
}

```

현재 MVP의 추천 로직은 rule-based scoring 방식을 사용한다. 백엔드 스키마 문서에서는 category 일치, price_level 일치, mood 태그 일치, parking 태그 일치, 평균 리뷰 신뢰도 점수를 기반으로 추천 점수를 계산하는 방식을 제시하고 있다.

6.8 Restaurant Detail API

6.8.1 Restaurant Detail Request

식당 상세 API는 사용자가 특정 식당 카드를 클릭하거나 지도 마커를 선택했을 때 호출된다.

Item	Description
Method	GET
Endpoint	/api/restaurants/{id}
Path Parameter	id: 식당 내부 ID
Authentication	Not required
Request Body	None
Response	식당 상세 정보

6.8.2 Response Example

```
{
  "id": 1,
  "campusId": 1,
  "name": "예시 파스타집",
  "category": "양식",
  "lat": 37.294,
  "lng": 126.975,
  "address": "경기도 수원시 장안구 예시로 1",
  "priceRange": "보통",
  "phone": "031-000-0001",
  "sourceUrl": "https://place.map.kakao.com/example1",
  "summary": "데이트 분위기와 파스타 언급이 많은 식당입니다.",
  "representativeMenu": "파스타|스테이크",
  "moodSummary": "데이트나 친구 모임에 적합합니다.",
  "parkingSummary": "주차 가능 여부에 대한 언급은 없습니다.",
  "waitingSummary": "저녁 시간대 웨이팅 언급이 일부 있습니다.",
  "averageTrustScore": 82.5,
```

```

"tags": [
  {
    "type": "mood",
    "value": "데이트",
    "confidenceScore": 0.9
  },
  {
    "type": "food",
    "value": "파스타",
    "confidenceScore": 0.95
  }
],
"hasAnalysis": true
}

```

6.9 LLM/Data Import Protocol

MVP 단계에서는 LLM/Data 팀이 생성한 CSV 파일을 백엔드가 import하여 데이터베이스에 저장한다. 이 방식은 백엔드가 사용자 요청마다 LLM을 실시간 호출하지 않고, 사전에 분석된 결과를 저장한 뒤 빠르게 제공할 수 있도록 한다.

현재 import 대상은 다음 세 가지 CSV이다.

File	Purpose	Key
restaurants.csv	식당 기본 정보 저장	external_id
restaurant_summaries.csv	식당별 요약 정보 저장	restaurant_external_id
restaurant_tags.csv	식당별 태그 정보 저장	restaurant_external_id

각 CSV는 external_id 또는 restaurant_external_id를 기준으로 연결된다.

restaurants.external_id는 외부 LLM/Data 결과와 백엔드 내부 식당 데이터를 연결하는 핵심 식별자이다.

Import 흐름은 다음과 같다.

LLM/Data CSV

→ *import_llm_data.py*

→ *Backend Database*

→ *FastAPI REST API*

→ *Frontend KakaoMap / RestaurantList / RestaurantDetail*

6.10 Admin and Analysis APIs

관리자 API는 일반 사용자 API와 분리되며, /api/admin prefix를 사용한다. 관리자 로그인 API를 제외한 모든 관리자 API는 Authorization: Bearer <token> 헤더를 필요로 한다. 현재 구현에서는 단일 관리자 계정을 환경 변수로 관리하고, 로그인 성공 시 JWT access token을 발급한다.

Method	Endpoint	Description	Status
POST	/api/admin/auth/log in	관리자 로그인	Implemented
POST	/api/admin/reviews/upload	리뷰 CSV 업로드	Implemented
POST	/api/admin/analysis-jobs	LLM 분석 작업 생성	Implemented
GET	/api/admin/analysis-jobs/{job_id}	분석 작업 상태 조회	Implemented
PUT	/api/admin/restaurants/{id}/summary	식당 요약 및 태그 수정	Planned
GET	/api/restaurants/{id}/review-score	식당 단위 리뷰 신뢰도 점수 조회	Planned
GET	/api/restaurants/{id}/trusted-reviews	신뢰 리뷰 후보 조회	Planned

분석 작업 상태는 다음 값 중 하나를 사용할 수 있다.

Status	Meaning
PENDING	분석 작업이 생성되었지만 아직 실행되지 않음
RUNNING	분석 진행 중
COMPLETED	분석 완료
FAILED	분석 실패
CANCELLED	작업 취소

향후 리뷰별 세부 분석 결과 API를 확장할 경우, 응답에는 원본 비협찬성 확률, 원본 리뷰 신뢰도 점수, 이벤트 규칙 적용 여부, 보정 후 확률, 보정 후 리뷰 신뢰도 점수, 모델 버전, 분석 시각을 포함할 수 있다. 현재 일반 사용자 API에서는 사용자 요청 시점에 LLM을 실시간 호출하지 않고, 사전에 저장된 식당 단위 요약과 Average Trust Score를 조회한다.

6.11 Authentication Protocol

일반 사용자 API는 로그인 없이 접근 가능하다. 일반 사용자는 캠퍼스 선택, 식당 목록 조회, 필터 기반 추천, 식당 상세 조회 기능을 사용할 수 있다.

반면 관리자 API는 인증된 관리자만 접근할 수 있다. 현재 구현에서는 MVP 범위에 맞춰 단일 관리자 계정을 환경 변수로 관리한다. 관리자가 /api/admin/auth/login에 username과 password를 전달하면, 백엔드는 환경 변수에 저장된 ADMIN_USERNAME, ADMIN_PASSWORD와 비교한다. 인증에 성공하면 JWT access token을 발급하며, 프론트엔드는 이후 관리자 API 요청에 Authorization: Bearer 헤더를 포함한다. 다중 관리자 계정과 역할 기반 권한 관리는 향후 admin_users 테이블을 추가하여 확장할 수 있다.

현재 구현에서는 JWT Bearer Token 방식을 사용한다. httpOnly Cookie Session 방식은 향후 보안 요구사항이 강화될 경우 검토할 수 있다.

Method	Description	Consideration
JWT Bearer Token	로그인 성공 시 access	구현이 단순하지만 token

token 발급 후 Authorization 저장 위치에 주의 필요
header에 포함

httpOnly Cookie Session	서버가 httpOnly cookie를 발급하고 브라우저가 자동 전송	XSS에 상대적으로 안전하지만 CSRF 대응 필요
-------------------------	---------------------------------------	-----------------------------

관리자 API 요청 예시는 다음과 같다.

POST /api/admin/auth/login

Content-Type: application/json

```
{  
  "username": "admin",  
  "password": "password"  
}
```

인증이 필요한 요청은 다음과 같이 token을 포함한다.

Authorization: Bearer <access_token>

6.12 Error Response Design

백엔드는 모든 API 오류를 일관된 JSON 형식으로 반환한다. 프론트엔드는 오류 응답을 바탕으로 사용자에게 적절한 안내 메시지를 표시한다.

공통 오류 응답 형식은 다음과 같다.

```
{  
  "success": false,  
  "errorCode": "RESTAURANT_NOT_FOUND",  
  "message": "식당 정보를 찾을 수 없습니다.",  
  "details": {  
    "restaurantId": 999  
  }  
}
```

주요 오류 유형은 다음과 같다.

Error Case	HTTP Status	Description
Campus Not Found	404	존재하지 않는 campus slug
Restaurant Not Found	404	존재하지 않는 restaurant id
Invalid Filter	400 또는 422	허용되지 않은 필터 조건
No Matching Restaurants	200 with empty list	조건에 맞는 식당 없음
Unauthorized Admin	401	인증되지 않은 관리자 요청
Forbidden Role	403	권한 없는 관리자 요청
Invalid CSV Format	400	CSV 필수 컬럼 누락 또는 형식 오류
Analysis Job Not Found	404	존재하지 않는 분석 작업
LLM Analysis Failed	500 또는 job status FAILED	분석 처리 실패
Internal Server Error	500	서버 내부 오류

“조건에 맞는 식당 없음”은 서버 오류가 아니므로 404보다는 빈 배열을 포함한 정상 응답으로 처리하는 것이 바람직하다.

```
{
  "campus": {
    "slug": "natural",
    "name": "성균관대학교 자연과학캠퍼스"
  },
  "restaurants": [],
  "message": "선택한 조건에 맞는 식당이 없습니다."
}
```

오류 응답에는 DB 접속 정보, API Key, stack trace, 내부 파일 경로와 같은 민감한 정보를 포함하지 않는다.

6.13 Protocol Design Constraints

본 시스템의 Protocol Design은 다음 제약사항을 따른다.

첫째, API 응답은 JSON 형식을 기본으로 한다.

둘째, 프론트엔드와 백엔드 사이의 외부 식별자는 가능한 한 campus_slug와 restaurant_id를 사용한다.

셋째, LLM/Data 결과와 DB 식당 데이터 연결에는 external_id를 사용한다.

넷째, 프론트엔드 응답 필드는 camelCase를 사용한다.

다섯째, 사용자 요청 시점에는 LLM을 실시간 호출하지 않고, 사전 저장된 분석 결과를 조회한다.

여섯째, 리뷰 신뢰도 점수는 참고 지표로만 제공하며, 리뷰의 진위 여부를 단정하는 표현을 사용하지 않는다.

일곱째, 관리자 API는 일반 사용자 API와 분리하며, 인증이 필요한 요청에는 Authorization: Bearer 헤더를 사용한다.

여덟째, 운영 환경에서는 HTTPS를 적용해야 한다.

아홉째, 지도 API Key, DB 접속 정보, LLM 모델 경로 등 민감하거나 환경에 따라 달라지는 값은 코드에 직접 작성하지 않는다.

6.14 Future Improvements

향후 Protocol Design에서는 리뷰별 세부 분석 결과 API, 다중 관리자 계정, 역할 기반 접근 제어, 관리자 변경 이력 조회 API를 확장할 수 있다. API 규모가 커질 경우 /api/v1/... 형태의 API versioning을 도입하고, 분석 작업의 실시간 진행률 제공이 중요해질 경우 WebSocket

또는 Server-Sent Events 방식을 검토한다. 자세한 향후 확장 항목은 10.5 Future Extension Items에서 정리한다.

Database Design

7.1 Objectives

Table Name	Purpose
campuses	서비스에서 지원하는 캠퍼스 구역 정보를 저장한다. natural, humanities와 같은 slug를 통해 프론트엔드 라우팅과 연결된다.
restaurants	식당명, 주소, 좌표, 카테고리, 가격대, 전화번호, 외부 식별자 등을 저장한다.
restaurant_summaries	식당별 AI 요약, 대표 메뉴, 분위기/주차/웨이팅 요약, 평균 리뷰 신뢰도 점수를 저장한다.
restaurant_tags	필터 검색과 추천 점수 계산에 사용할 식당별 태그를 저장한다.
reviews	관리자 업로드를 통해 저장되는 식당별 원본 리뷰 데이터를 저장한다.
analysis_jobs	관리자 분석 작업의 상태, 진행률, 완료 시각, 오류 메시지를 저장한다.

본 장에서는 **LLM 기반 리뷰 신뢰도 분석 맛집 추천 시스템**에서 사용하는 데이터베이스 구조를 설명한다. 데이터베이스는 캠퍼스 정보, 식당 기본 정보, 식당별 AI 요약, 추천 및 필터링에 사용되는 태그 정보를 저장하며, 백엔드 API가 프론트엔드에 제공하는 데이터의 기반이 된다.

현재 구현된 데이터베이스는 사용자용 식당 탐색 기능을 위한 네 개의 핵심 테이블과 관리자 연동을 위한 두 개의 보조 테이블로 구성된다. 사용자 기능의 핵심 테이블은 campuses, restaurants, restaurant_summaries, restaurant_tags이며, 관리자 리뷰 업로드와 분석 작업 상태 관리를 위해 reviews와 analysis_jobs를 함께 사용한다. 이 구조는 지도 기반 탐색, 리스트 기반 탐색, 식당 상세 조회, 필터 기반 추천 기능을 지원하기 위한 최소 데이터베이스 구조이다.

향후 확장 단계에서는 리뷰별 세부 분석 결과 저장, 식당 단위 분석 통계 분리, 다중 관리자 계정 관리, 관리자 변경 이력 저장을 위해 `review_analyses`, `restaurant_analyses`, `admin_users`, `admin_change_logs` 테이블을 추가할 수 있다.

7.2 Current Database Overview

현재 구현 데이터베이스는 다음 관계를 가진다.

Campus 1 ——— N Restaurant

Restaurant 1 ——— 1 RestaurantSummary

Restaurant 1 ——— N RestaurantTag

Restaurant 1 ——— N Review

Restaurant 1 ——— N AnalysisJob

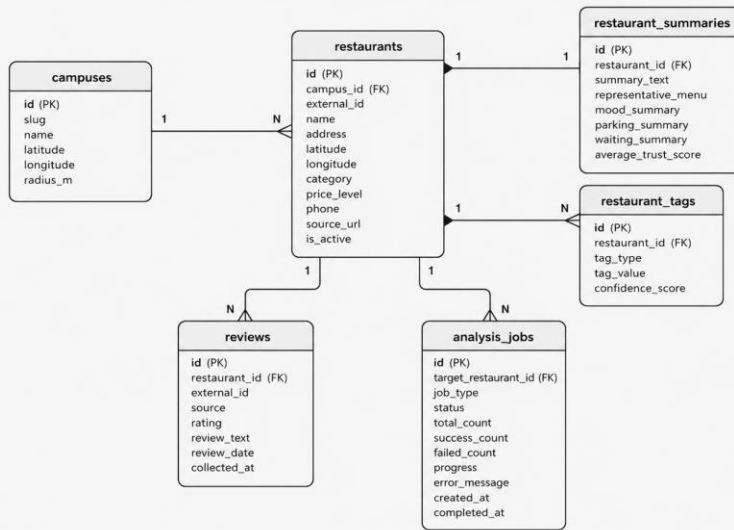
하나의 캠퍼스는 여러 식당을 가질 수 있다. 각 식당은 하나의 요약 정보를 가지며, 여러 개의 태그를 가질 수 있다. 또한 관리자가 업로드한 여러 리뷰 원문은 하나의 식당에 연결되고, 하나의 식당에 대해 여러 분석 작업이 생성될 수 있다. 이 구조는 일반 사용자 화면의 식당 탐색 및 추천 기능뿐 아니라 관리자 화면의 리뷰 업로드와 분석 작업 상태 관리 기능까지 지원한다.

현재 개발 단계에서는 SQLite를 사용하지만, schema는 PostgreSQL 전환을 고려하여 설계한다. 따라서 최종 구현 단계에서는 DATABASE_URL을 환경 변수로 분리하고 PostgreSQL 기반 운영 DB로 전환할 수 있도록 한다.

7.3 ER Diagram

본 시스템의 ER Diagram은 사용자 기능의 핵심 테이블과 관리자 연동 테이블을 함께 포함한다.

Figure 7.1 ER Diagram



ER Diagram은 현재 구현 데이터베이스의 주요 테이블과 관계를 나타낸다. restaurants 테이블은 campuses와 연결되며, restaurant_summaries, restaurant_tags, reviews, analysis_jobs의 중심 테이블로 동작한다. 이 구조는 사용자 식당 탐색 기능과 관리자 리뷰 업로드 및 분석 작업 상태 관리 기능을 함께 지원한다.

7.4 Table Descriptions

7.4.1 campuses

campuses 테이블은 서비스에서 지원하는 캠퍼스 권역 정보를 저장한다. 현재 시스템은 성균관대학교 자연과학캠퍼스와 인문사회캠퍼스를 대상으로 한다.

Column	Type	Description
id	Integer	캠퍼스 내부 ID
slug	String	프론트엔드 라우팅 및 API에서 사용하는 캠퍼스 식별자
name	String	캠퍼스 이름
latitude	Float	캠퍼스 중심 위도
longitude	Float	캠퍼스 중심 경도

radius_m	Integer	캠퍼스 주변 탐색 반경
----------	---------	--------------

slug는 프론트엔드의 /campus/:slug 라우팅과 백엔드 API를 연결하는 핵심 필드이다. 예를 들어 자연과학캠퍼스는 natural, 인문사회캠퍼스는 humanities와 같은 값을 가진다.

7.4.2 restaurants

restaurants 테이블은 식당의 기본 정보를 저장한다. 지도 마커 표시, 식당 리스트 카드, 식당 상세 페이지의 기본 데이터는 이 테이블을 기반으로 제공된다.

Column	Type	Description
id	Integer	식당 내부 ID
campus_id	Integer	소속 캠퍼스 ID
external_id	String	LLM/Data 또는 외부 API 기준 식당 ID
name	String	식당명
address	String	식당 주소
latitude	Float	식당 위도
longitude	Float	식당 경도
category	String	음식 종류
price_level	String	가격대
phone	String	전화번호
source_url	String	카카오맵 등 원본 링크
is_active	Boolean	사용자 화면 노출 여부

restaurants.id는 백엔드 내부 식별자이고, restaurants.external_id는 LLM/Data 결과, 외부 CSV 데이터, 리뷰 업로드 CSV의 restaurant_external_id와 내부 식당 데이터를 연결하는 외부 식별자이다. 관리자가 리뷰 CSV를 업로드할 때 백엔드는 restaurant_external_id를 기준으로 기존 식당을 찾고, 해당 식당의 id를 reviews.restaurant_id에 저장한다.

is_active는 식당 데이터의 활성 상태를 나타낸다. 관리자 삭제 기능은 실제 레코드를 삭제하지 않고 is_active 값을 false로 변경하는 soft delete 방식으로 처리한다. 이를 통해 기존 리뷰, 요약, 태그, 분석 작업과의 관계를 유지하면서 사용자 화면에서는 비활성 식당을 제외할 수 있다.

7.4.3 restaurant_summaries

restaurant_summaries 테이블은 식당 단위의 LLM/Data 요약 결과를 저장한다. 이 테이블의 정보는 식당 리스트와 상세 페이지에서 사용자에게 제공된다.

Column	Type	Description
id	Integer	요약 내부 ID
restaurant_id	Integer	연결된 식당 ID
summary_text	Text	식당 전체 요약
representative_menu	String	대표 메뉴
mood_summary	String	분위기 요약
parking_summary	String	주차 관련 요약
waiting_summary	String	웨이팅 관련 요약
average_trust_score	Float	평균 리뷰 신뢰도 점수

현재 구현에서는 average_trust_score가 식당별 리뷰 신뢰도 경향을 나타내는 간단한 점수로 사용된다. 다만 최종 설계에서는 요약 정보와 분석 통계를 분리하기 위해, 식당 단위 분석 점수는 별도의 restaurant_analyses 테이블로 분리하는 것이 바람직하다.

7.4.4 restaurant_tags

restaurant_tags 테이블은 필터 검색과 추천 점수 계산에 사용할 태그 정보를 저장한다.

Column	Type	Description
id	Integer	태그 내부 ID

restaurant_id	Integer	연결된 식당 ID
tag_type	String	태그 유형
tag_value	String	태그 값
confidence_score	Float	태그 신뢰도

태그 유형과 값의 예시는 다음과 같다.

tag_type	tag_value 예시	사용 목적
mood	데이트, 조용함, 캐주얼	분위기 필터
food	파스타, 국밥, 분식	음식 종류/대표 메뉴
price	저렴함, 보통, 비쌘	가격대 필터
purpose	혼밥, 모임, 데이트	방문 목적
parking	주차가능, 주차불가	주차 여부 필터
waiting	웨이팅있음, 웨이팅적음	웨이팅 정보

이 구조를 사용하면 특정 필터 조건과 식당 태그를 비교하여 추천 점수를 계산할 수 있다. 또한 태그에 confidence_score를 함께 저장하여, LLM 또는 데이터 관리자가 생성한 태그의 신뢰도를 추천 점수에 반영할 수 있다.

7.5 Relational Schema

현재 구현 데이터베이스의 관계형 스키마는 다음과 같이 정리할 수 있다.

```
campuses(
  id PK,
  slug UNIQUE NOT NULL,
  name NOT NULL,
  latitude,
  longitude,
```

radius_m
)

restaurants(
id PK,
campus_id FK -> campuses.id,
external_id UNIQUE,
name NOT NULL,
address,
latitude,
longitude,
category,
price_level,
phone,
source_url,
is_active
)

restaurant_summaries(
id PK,
restaurant_id FK -> restaurants.id,
summary_text,
representative_menu,
mood_summary,
parking_summary,
waiting_summary,
average_trust_score
)

restaurant_tags(
id PK,
restaurant_id FK -> restaurants.id,

```

tag_type NOT NULL,
tag_value NOT NULL,
confidence_score
)

reviews(
  id PK,
  restaurant_id FK -> restaurants.id,
  external_id,
  source,
  rating,
  review_text,
  review_date,
  collected_at
)

analysis_jobs(
  id PK,
  job_type,
  status,
  target_restaurant_id FK -> restaurants.id,
  total_count,
  success_count,
  failed_count,
  progress,
  error_message,
  created_at,
  completed_at
)

```

주요 관계는 다음과 같다.

Relationship

Description

campuses.id → restaurants.campus_id	하나의 캠퍼스는 여러 식당을 가질 수 있음
restaurants.id → restaurant_summaries.restaurant_id	하나의 식당은 하나의 요약 정보를 가짐
restaurants.id → restaurant_tags.restaurant_id	하나의 식당은 여러 태그를 가질 수 있음
restaurants.id → reviews.restaurant_id	하나의 식당은 여러 리뷰 원문을 가질 수 있음
restaurants.id → analysis_jobs.target_restaurant_id	하나의 식당에 대해 여러 분석 작업이 생성될 수 있음

7.6 LLM/Data Import Schema

MVP 단계에서 LLM/Data 결과는 CSV 파일 형태로 백엔드 DB에 import된다. 백엔드는 import_llm_data.py를 통해 CSV를 읽고, 각 테이블에 데이터를 저장한다.

CSV File	Purpose	Key Link
restaurants.csv	식당 기본 정보 저장	external_id
restaurant_summaries.csv	식당별 요약 정보와 Average Trust Score 저장	restaurant_external_id → restaurants.external_id
restaurant_tags.csv	식당별 태그 정보 저장	restaurant_external_id → restaurants.external_id

restaurants.csv는 식당 기본 정보를 저장한다. 주요 필드는 external_id, campus_slug, name, address, lat, lng, category, price_range, source_url, phone이다.

restaurant_summaries.csv는 식당별 요약 정보와 Average Trust Score를 저장한다. 주요 필드는 restaurant_external_id, summary, representative_menu, mood_summary, parking_summary, waiting_summary, average_trust_score, has_analysis이다.

restaurant_tags.csv는 식당별 태그 정보를 저장한다. 주요 필드는 restaurant_external_id, tag_type, tag_value, confidence_score이다.

이 구조에서 중요한 점은 백엔드 내부 식당 ID가 아니라 external_id를 기준으로 LLM/Data 결과를 연결한다는 것이다. restaurants.csv는 external_id를 식당의 외부 식별자로 사용하고, restaurant_summaries.csv와 restaurant_tags.csv는 restaurant_external_id를 통해 restaurants.external_id와 연결된다. 이를 통해 LLM/Data 파트와 백엔드 파트가 독립적으로 작업하면서도 동일한 식당 데이터를 안정적으로 연결할 수 있다.

7.9 Future Schema Extension

현재 구현에서는 식당 단위 요약, 태그, 리뷰 원문, 분석 작업 상태를 우선 연결한다. 향후 확장 단계에서는 리뷰별 세부 분석 결과, 식당 단위 분석 통계, 다중 관리자 계정, 관리자 변경 이력 관리를 위해 추가 테이블을 도입할 수 있다.

Future Table	Purpose	Key Columns
review_analyses	리뷰별 LLM/Data 분석 결과 저장	id, review_id, trust_score, ad_probability, label, sentiment, keywords, extracted_info, analyzed_at
restaurant_analyses	식당 단위 리뷰 신뢰도 통계 저장	restaurant_id, review_count, average_review_credibility, low_credibility_review_ratio, high_credibility_review_ratio, restaurant_review_reliability_score, model_version, updated_at
admin_users	다중 관리자 계정 및 역할 관리	id, email, password_hash, role
admin_change_logs	관리자 수정 이력 관리	id, admin_user_id, action_type, target_table, target_id, before_data, after_data

향후 review_analyses 테이블은 모델 원본 점수, 후처리 적용 여부, 보정 후 점수, 모델 버전, 분석 시각을 저장할 수 있도록 확장한다. 이를 통해 리뷰별 분석 결과와 모델 변경 이력을 분리하여 관리할 수 있다. review_analyses 테이블은 리뷰별 분석 결과를 식당 단위로 집계하여 평균 리뷰 신뢰도 점수와 식당 단위 분석 통계를 관리하는 데 사용한다.

7.10 Database Design Constraints

본 시스템의 데이터베이스 설계는 다음 제약사항을 따른다.

첫째, 프론트엔드 라우팅과 API 연결을 위해 `campuses.slug`를 사용한다. `slug`는 `unique`해야 하며, `natural`, `humanities`와 같이 사람이 이해할 수 있는 값으로 관리한다.

둘째, LLM/Data 결과와 백엔드 DB의 식당 데이터를 연결하기 위해 `restaurants.external_id`를 사용한다. `external_id`는 외부 데이터와의 연결 키이므로 중복되지 않도록 관리해야 한다.

셋째, 지도 기반 탐색을 위해 `restaurants.latitude`와 `restaurants.longitude`를 저장한다. 좌표가 없는 식당은 지도 마커로 표시할 수 없으므로, 리스트 기반 탐색에서만 표시하거나 관리자 검토 대상으로 분리할 수 있다.

넷째, 식당 요약과 태그는 사용자 화면에 직접 노출될 수 있으므로, 부적절하거나 단정적인 표현이 포함되지 않도록 관리해야 한다.

다섯째, 리뷰 신뢰도 점수는 리뷰의 실제 진위 여부를 단정하는 값이 아니라 참고 지표이므로, DB 필드명과 사용자 화면 표현에서도 `fake_review_score`와 같은 단정적 이름을 사용하지 않는다.

여섯째, 향후 `admin_users` 테이블을 도입하는 경우, 관리자 비밀번호는 평문으로 저장하지 않고 `password_hash` 형태로 저장한다.

일곱째, 향후 `admin_change_logs` 테이블을 도입하는 경우, 관리자에 의한 식당 정보, 요약, 태그 수정 이력을 저장하여 변경 내역을 추적할 수 있어야 한다.

여덟째, 운영 환경에서는 자주 조회되는 컬럼에 인덱스를 설정해야 한다. 우선적으로 `campuses.slug`, `restaurants.campus_id`, `restaurants.external_id`, `restaurant_tags.restaurant_id`, `restaurant_tags.tag_type`, `restaurant_tags.tag_value`에 대한 인덱스를 고려한다.

Testing Plan

8.1 Objectives

본 장에서는 SKKU Gourmap의 테스트 계획을 설명한다. 본 시스템은 Frontend Web Client, Backend API Server, Database, LLM/Data Analysis Module, Kakao Map API가 함께 동작하는 구조이므로, 각 모듈의 단위 기능뿐만 아니라 모듈 간 연동과 전체 사용자 흐름을 함께 검증해야 한다.

테스트의 주요 목적은 다음과 같다.

첫째, 요구사항 명세서에서 정의한 기능이 실제 시스템에서 올바르게 동작하는지 확인한다.

둘째, 캠퍼스 선택, 식당 목록 조회, 지도 및 리스트 탐색, 필터 기반 추천, 식당 상세 조회 등 일반 사용자 기능을 검증한다.

셋째, 리뷰 신뢰도 점수, 식당 요약, 태그, 추천 근거가 사용자에게 일관된 형태로 제공되는지 확인한다.

넷째, 백엔드 API, 데이터베이스, LLM/Data import 결과가 올바르게 연결되는지 확인한다.

다섯째, Kakao Map API 실패, 데이터 없음, 필터 결과 없음, 분석 정보 없음과 같은 예외 상황에서도 사용자가 서비스를 계속 이용할 수 있는지 검증한다.

여섯째, 현재 구현된 관리자 인증, 리뷰 CSV 업로드, 분석 작업 상태 조회 기능이 안전하게 동작하는지 확인한다. 요약 및 태그 수정, 다중 관리자 계정, 역할 기반 권한 분리는 향후 확장 단계에서 추가 검증한다.

8.2 Testing Strategy

본 시스템의 테스트는 **Development Testing, Integration Testing, System Testing, User Testing**의 순서로 진행한다.

Test Type	Purpose	Main Target
Unit Testing	개별 함수와 모듈의 정상 동작 확인	추천 점수 계산, CSV import, 데이터 변환 함수
Component Testing	개별 화면 또는 API 단위 검증	CampusSelectPage, KakaoMap, Restaurant API
Integration Testing	프론트엔드-백엔드-DB 연동 확인	식당 목록 조회, 추천 결과 조회, 상세 조회
System Testing	전체 사용자 흐름 검증	캠퍼스 선택부터 식당 상세 확인까지
User Testing	실제 사용자 관점의 사용성 검증	Alpha, Beta, Acceptance Test

현재 MVP 단계에서는 캠퍼스 조회, 식당 목록 조회, 식당 상세 조회, 필터 기반 추천, 지도/리스트 UI, CSV 기반 데이터 import를 우선 테스트한다. 현재 구현된 관리자 인증, 리뷰 CSV 업로드, 분석 작업 생성 및 상태 조회 기능도 함께 검증한다. 리뷰별 세부 분석 결과 저장, 다중 관리자 계정, 관리자 변경 이력 저장은 향후 확장 단계에서 추가 테스트한다.

8.3 Frontend Testing

프론트엔드 테스트는 사용자가 실제로 접하는 화면과 상호작용을 중심으로 수행한다. 주요 테스트 대상은 캠퍼스 선택 화면, 지도+리스트 탐색 화면, 리스트 전용 화면, 식당 상세 화면, 관리자 화면이다.

프론트엔드 테스트 항목은 다음과 같다.

Test Target	Test Content
CampusSelectPage	자연과학캠퍼스와 인문사회캠퍼스를 선택했을 때 올바른

	URL로 이동하는지 확인
HomePage	선택된 캠퍼스의 지도와 식당 리스트가 표시되는지 확인
ListOnlyPage	지도 없이도 식당 목록이 정상적으로 표시되는지 확인
RestaurantDetailPage	식당 상세 정보, 요약, 태그, 신뢰도 점수가 표시되는지 확인
KakaoMap	지도 로딩, 중심 좌표 설정, 마커 표시가 정상 동작하는지 확인
RestaurantCard	식당명, 카테고리, 요약, 태그, 추천 점수가 올바르게 표시되는지 확인
ErrorMessage	API 실패 또는 데이터 없음 상황에서 적절한 오류 메시지가 표시되는지 확인

특히 Kakao Map API는 외부 서비스에 의존하므로, 지도 로딩 실패 상황을 반드시 테스트한다. 지도 API가 실패하더라도 사용자는 `/campus/:slug/list` 경로를 통해 리스트 기반으로 식당 정보를 탐색할 수 있어야 한다.

프론트엔드 테스트는 실제 백엔드 API 응답을 기준으로 수행한다. 특히 FilterPanel, 관리자 라우트 보호, 분석 작업 상태 polling, 지도 API 실패 fallback은 일반 사용자 흐름과 관리자 흐름에서 각각 검증한다.

8.4 Backend API Testing

백엔드 API 테스트는 FastAPI endpoint가 올바른 요청을 받고, 기대한 JSON 응답을 반환하는지 확인하는 방식으로 수행한다.

현재 MVP에서 우선 테스트할 API는 다음과 같다.

API	Test Content
GET <code>/api/campuses</code>	캠퍼스 목록이 정상적으로 반환되는지 확인
GET <code>/api/campuses/{slug}/restaurants</code>	특정 캠퍼스의 식당 목록이 반환되는지 확인
GET <code>/api/campuses/{slug}/recommendations</code>	필터 조건에 따라 추천 점수와 추천 근거가 반환되는지 확인

GET /api/restaurants/{id}

특정 식당의 상세 정보가 반환되는지 확인

API 테스트에서는 정상 입력뿐 아니라 잘못된 입력도 함께 검증한다. 예를 들어 존재하지 않는 campus_slug, 존재하지 않는 restaurant_id, 허용되지 않는 필터 값이 들어왔을 때 적절한 오류 응답이 반환되어야 한다.

대표적인 오류 테스트는 다음과 같다.

Error Case	Expected Result
존재하지 않는 캠퍼스 slug	404 Not Found
존재하지 않는 식당 ID	404 Not Found
잘못된 필터 값	400 또는 422 응답
조건에 맞는 식당 없음	빈 배열과 안내 메시지 반환
서버 내부 오류	민감한 정보 없이 일반 오류 메시지 반환

“조건에 맞는 식당 없음”은 시스템 오류가 아니므로, 빈 리스트를 포함한 정상 응답으로 처리하는 것이 바람직하다.

8.5 Database and Data Import Testing

Database 테스트는 식당 데이터, 요약 데이터, 태그 데이터, 리뷰 업로드 데이터, 분석 작업 상태 데이터가 올바르게 저장되고 조회되는지 확인하는 것을 목표로 한다. MVP 사용자 기능의 핵심 테스트 대상은 campuses, restaurants, restaurant_summaries, restaurant_tags이다. 현재 구현된 관리자 연동 기능에서는 reviews와 analysis_jobs도 함께 검증한다.

주요 테스트 항목은 다음과 같다.

Test Target

Test Content

Campus Data	natural, humanities 캠퍼스가 올바른 campus_slug와 좌표를 가지는지 확인
Restaurant Data	식당명, 주소, 좌표, 카테고리, 가격대가 정상 저장되는지 확인
RestaurantSummary	식당별 요약, 대표 메뉴, 분위기, 주차, 웨이팅 요약이 연결되는지 확인
RestaurantTag	식당별 태그가 여러 개 저장되고 조회되는지 확인
Review Data	관리자 리뷰 CSV 업로드 결과가 식당과 올바르게 연결되는지 확인
AnalysisJob Data	분석 작업 상태가 PENDING, RUNNING, COMPLETED, FAILED로 저장되고 조회되는지 확인
External ID Mapping	LLM/Data 결과의 external_id가 실제 식당과 올바르게 연결되는지 확인

CSV import 테스트에서는 필수 컬럼 누락, 잘못된 campus_slug, 존재하지 않는 restaurant_external_id, 좌표 누락, 중복 external_id 등의 상황을 확인한다. 잘못된 데이터가 들어온 경우 전체 시스템이 중단되지 않고, 오류 행을 식별하여 관리자에게 알려줄 수 있어야 한다.

8.6 Recommendation Testing

추천 테스트는 필터 조건과 식당 태그, 평균 신뢰도 점수에 따라 추천 결과가 일관되게 생성되는지 확인한다.

현재 MVP 추천 로직은 rule-based scoring 방식이므로, 같은 입력에 대해 항상 같은 추천 결과가 나와야 한다. 테스트에서는 다음 조건을 확인한다.

Test Case	Expected Result
category가 일치하는 식당	추천 점수에 category 점수 반영
price_level이 일치하는 식당	추천 점수에 price 점수 반영
mood 태그가 일치하는 식당	추천 점수에 mood 점수 반영

parking 태그가 일치하는 식당	추천 점수에 parking 점수 반영
평균 리뷰 신뢰도 점수가 높은 식당	추천 점수에 신뢰도 점수 반영
여러 조건이 동시에 일치하는 식당	추천 순위가 상위에 위치
조건에 맞는 식당이 없는 경우	빈 결과와 안내 메시지 반환

추천 테스트에서 중요한 점은 추천 점수만 확인하는 것이 아니라, recommendationReasons가 실제 점수 계산 근거와 일치하는지 확인하는 것이다. 예를 들어 주차 필터가 적용되지 않았는데 “주차 가능성이 높습니다”라는 추천 근거가 표시되어서는 안 된다.

8.7 Review Credibility and LLM/Data Testing

리뷰 신뢰도 분석 테스트는 모델 출력값과 후처리 규칙이 의도한 방식으로 동작하는지 확인한다. 본 시스템에서 리뷰 신뢰도 점수는 리뷰의 실제 진위 여부를 단정하는 값이 아니라, 비협찬성 가능성과 이벤트 영향 가능성을 나타내는 참고 지표로 사용된다.

주요 테스트 항목은 다음과 같다.

Test Target	Test Content
Score Range	리뷰 신뢰도 점수가 0~100 범위 안에 있는지 확인
Event Keyword Rule	“리뷰 이벤트”, “사진 리뷰”, “별 다섯” 등 명시적 이벤트 문구가 감지되는지 확인
Score Calibration	이벤트 문구가 있는 경우 점수가 상한선 이하로 보정되는지 확인
Neutral Expression	사용자 화면에서 “가짜 리뷰”, “조작 리뷰” 같은 단정적 표현을 사용하지 않는지 확인
Model Metadata Extension	향후 리뷰별 세부 분석 결과 저장 시 모델 버전과 분석 시각이 기록되는지 확인

예를 들어 “사진 리뷰 남기면 서비스 주신대서 작성합니다”와 같은 리뷰는 이벤트 문구가 감지되어야 하며, 모델 원본 점수가 높더라도 최종 신뢰도 점수는 보정되어야 한다.

8.8 Admin and Security Testing

관리자 기능은 식당 정보, 리뷰 데이터, 분석 결과, 요약 및 태그를 관리하는 기능이므로 일반 사용자 기능보다 더 엄격한 테스트가 필요하다. 관리자 기능 테스트는 현재 구현된 관리자 인증, 식당 관리, 리뷰 CSV 업로드, 분석 작업 상태 조회를 대상으로 수행한다. 다중 관리자 계정, 역할 기반 권한 분리, 관리자 변경 이력 저장은 향후 확장 기능으로 분리하여 추가 테스트한다.

관리자 기능 테스트 항목은 다음과 같다.

Test Target	Scope	Test Content
Admin Login	Current	올바른 계정으로 로그인 성공 여부 확인
Invalid Login	Current	잘못된 계정 또는 비밀번호 입력 시 로그인 실패 확인
AdminGuard	Current	인증되지 않은 사용자가 관리자 페이지에 접근할 수 없는지 확인
Review Upload	Current	리뷰 CSV 업로드와 형식 검증이 정상 동작하는지 확인
Analysis Job Status	Current	분석 작업 상태가 PENDING, RUNNING, COMPLETED, FAILED로 갱신되는지 확인
Summary/Tag Edit	Future	관리자가 요약과 태그를 수정하면 사용자 화면에 반영되는지 확인
Role-based Access	Future	관리자 역할에 따라 접근 가능한 기능이 제한되는지 확인
Change Log	Future	관리자 수정 이력이 저장되는지 확인

보안 테스트에서는 비밀번호가 평문으로 저장되지 않는지, 인증 토큰이 필요한 API에 인증 없이 접근할 수 없는지, 서버 오류 응답에 DB 접속 정보나 내부 경로가 노출되지 않는지 확인한다.

8.9 Requirements-based Test Cases

본 절에서는 주요 기능 요구사항에 대한 테스트 케이스를 정리한다.

Requirement	Test Scenario	Expected Result
FR-01	사용자가 자연과학캠퍼스를 선택한다	/campus/natural로 이동한다
FR-01	사용자가 인문사회캠퍼스를 선택한다	/campus/humanities로 이동한다
FR-02	선택한 캠퍼스의 식당 목록을 요청한다	해당 캠퍼스 식당 목록이 반환된다
FR-03	지도 화면을 연다	캠퍼스 중심 지도와 식당 마커가 표시된다
FR-04	리스트 화면을 연다	식당 카드 목록이 표시된다
FR-05	특정 식당을 선택한다	식당 상세 정보가 표시된다
FR-06	필터 조건을 선택한다	선택한 필터 상태가 반영된다
FR-07	필터 기반 추천을 요청한다	추천 점수 기준으로 정렬된 결과가 반환된다
FR-08	추천 결과를 확인한다	추천 근거가 함께 표시된다
FR-12	관리자가 로그인한다	인증 성공 시 관리자 대시보드로 이동한다
FR-13	관리자가 식당 정보를 수정한다	수정 내용이 DB와 사용자 화면에 반영된다
FR-14	관리자가 리뷰 데이터를	유효한 리뷰 데이터가 저장된다

업로드한다

FR-15	관리자가 분석 결과를 확인한다	분석 상태와 결과가 표시된다
FR-16	지도 API가 실패한다	리스트 기반 탐색 화면이 제공된다

8.10 User Testing

사용자 테스트는 실제 사용자의 관점에서 시스템이 이해하기 쉽고 유용한지 확인하는 과정이다. 본 시스템에서는 Alpha Test, Beta Test, Acceptance Test를 단계적으로 수행한다.

8.10.1 Alpha Test

Alpha Test는 개발팀 내부에서 수행한다. 팀원들은 일반 사용자와 관리자 역할을 나누어 캠퍼스 선택, 식당 탐색, 필터 적용, 상세 정보 확인, 지도 실패 상황, 데이터 없음 상황을 테스트한다. 이 단계에서는 기능 오류, 화면 깨짐, 잘못된 추천 근거, API 오류 처리를 중심으로 확인한다.

8.10.2 Beta Test

Beta Test는 실제 서비스 대상과 유사한 사용자에게 제한적으로 제공하여 수행한다. 예를 들어 성균관대학교 학생 몇 명에게 시스템을 사용하게 한 뒤, 식당 탐색 흐름이 직관적인지, 추천 근거가 이해되는지, 리뷰 신뢰도 점수 표현이 오해를 만들지 않는지 확인한다.

8.10.3 Acceptance Test

Acceptance Test는 시스템이 최종 요구사항을 만족하는지 확인하는 테스트이다. 캠퍼스 선택부터 식당 추천, 식당 상세 조회, 지도 실패 시 리스트 fallback, 관리자 기능의 기본 흐름이 요구사항에 맞게 동작하면 통과로 본다.

Acceptance Test의 기준은 다음과 같다.

Acceptance Criteria

캠퍼스 선택 가능

Pass Condition

두 캠퍼스 중 하나를 선택해 탐색 화면으로 이동 가능

식당 탐색 가능	지도 또는 리스트에서 식당 정보 확인 가능
필터 추천 가능	필터 조건에 따라 추천 결과와 추천 근거 제공
상세 정보 확인 가능	식당 요약, 태그, 신뢰도 점수 확인 가능
오류 대응 가능	지도 실패나 데이터 없음 상황에서 대체 안내 제공
중립적 표현 유지	리뷰를 가짜 또는 조작으로 단정하지 않음
관리자 기능 보호	관리자 기능은 인증된 사용자만 접근 가능

8.11 Test Environment

테스트 환경은 개발 환경과 배포 예정 환경을 구분하여 구성한다.

Environment	Description
Frontend Local	Vite 개발 서버에서 React UI 테스트
Backend Local	FastAPI 개발 서버에서 API 테스트
Database Local	SQLite 또는 PostgreSQL 개발 DB 사용
Browser	Chrome 기준 기능 및 UI 확인
API Test Tool	FastAPI Swagger UI 또는 Postman 사용
Data Import Test	CSV import script를 이용한 샘플 데이터 적재
Map API Test	Kakao Map API Key와 허용 도메인 설정 확인

개발 단계에서는 SQLite로 빠르게 기능을 확인할 수 있지만, 최종 통합 테스트에서는 PostgreSQL 환경에서 동일한 API와 데이터 조회가 정상 동작하는지 확인해야 한다.

Development Plan

9.1 Objectives

본 장에서는 LLM 기반 리뷰 신뢰도 분석 맛집 추천 시스템의 개발 계획을 설명한다. 본 시스템은 React 기반 프론트엔드, FastAPI 기반 백엔드, SQLAlchemy 기반 데이터베이스 접근 구조, LLM/Data 분석 결과를 활용하는 데이터 import 구조로 구성된다.

본 장의 목적은 프론트엔드, 백엔드, 데이터베이스, LLM/Data 모듈의 개발 환경과 구현 우선순위를 정리하는 것이다. 또한 환경 변수, 외부 API Key, 모델 파일, 데이터 파일 관리 방식을 명확히 하여 팀원 간 협업 과정에서 발생할 수 있는 인터페이스 불일치를 줄이는 것을 목표로 한다.

세부 프론트엔드 구조는 4장, 백엔드 구조는 5장, API 형식은 6장, 데이터베이스 구조는 7장에서 설명하므로, 본 장에서는 각 모듈을 어떤 순서로 개발하고 통합할 것인지에 초점을 둔다.

9.2 Development Environment

본 시스템의 개발 환경은 프론트엔드, 백엔드, 데이터베이스, LLM/Data 분석 환경으로 구분된다.

Area	Technology
Frontend	React 18, TypeScript, Vite, React Router v6, Axios, CSS Modules
Backend	Python, FastAPI, SQLAlchemy

Development DB	SQLite
Target DB	PostgreSQL
Map API	Kakao Map JavaScript API
LLM Model	A.X Lite 기반 LoRA fine-tuned model
Data Import	CSV 기반 import script
Version Control	Git, GitHub
API Test	FastAPI Swagger UI, Postman
Browser Test	Chrome 기준 테스트

개발 초기에는 SQLite를 사용하여 빠르게 기능을 구현하고 테스트한다. 이후 최종 통합 단계에서는 PostgreSQL 환경으로 전환하여 실제 운영 환경과 유사한 조건에서 API, 데이터 조회, 추천 기능이 정상적으로 동작하는지 확인한다. 외부 API Key, DB 접속 정보, JWT secret, LLM 모델 경로와 같이 환경에 따라 달라지는 값은 코드에 직접 작성하지 않고 환경 변수로 관리한다.

9.3 Frontend Development Plan

프론트엔드는 일반 사용자 화면과 관리자 화면으로 나누어 개발한다. 일반 사용자 화면은 캠퍼스 선택, 지도 기반 탐색, 리스트 기반 탐색, 필터 기반 추천, 식당 상세 조회를 제공한다. 관리자 화면은 로그인, 대시보드, 식당 관리, 리뷰 관리, 분석 작업 상태 확인 기능을 제공하는 방향으로 구성한다.

프론트엔드 개발의 핵심은 백엔드 API 응답과 화면 상태를 일관되게 연결하는 것이다. 특히 캠퍼스 선택 결과는 campus_slug를 기준으로 라우팅과 API 요청에 반영되어야 하며, 식당 목록과 추천 결과는 지도 마커와 리스트 카드에 동일한 데이터 원천으로 표시되어야 한다.

Priority	Task
1	API endpoint와 프론트엔드 service layer 연결
2	캠퍼스별 식당 목록을 지도와 리스트에 동시 표시
3	FilterPanel과 추천 API 연동
4	식당 상세 페이지를 실제 API 응답 기반으로 정리
5	Kakao Map 로딩 실패 시 리스트 fallback 제공
6	관리자 로그인 및 관리자 라우트 보호 적용
7	관리자 식당 관리, 리뷰 업로드, 분석 작업 상태 화면 연동
8	모바일 반응형 UI와 공통 오류 처리 컴포넌트 개선

9.4 Backend Development Plan

백엔드는 FastAPI 기반 REST API 서버로 개발한다. 현재 구현 범위에서는 캠퍼스 목록 조회, 캠퍼스별 식당 목록 조회, 식당 상세 조회, 필터 기반 추천 API를 우선 안정화한다. 또한 관리자 화면 연동을 위해 관리자 인증, 식당 관리, 리뷰 CSV 업로드, 분석 작업 생성 및 상태 조회 기능을 제공한다.

백엔드 개발의 핵심은 프론트엔드 응답 schema와 데이터베이스 schema를 일관되게 유지하는 것이다. DB 내부에서는 snake_case 필드를 사용하고, 프론트엔드 응답에서는 camelCase 필드를 사용하므로, 응답 변환 로직과 TypeScript 타입이 일치해야 한다.

Priority	Task
1	일반 사용자용 MVP API 안정화
2	프론트엔드 응답 schema와 백엔드 DTO 정렬
3	추천 로직을 별도 service layer로 분리
4	CSV import 기능과 external_id 매칭 검증

- | | |
|---|--|
| 5 | 관리자 인증, 식당 관리, 리뷰 업로드 API 안정화 |
| 6 | 분석 작업 상태 관리와 polling 응답 구조 정리 |
| 7 | SQLite 개발 DB에서 PostgreSQL 운영 DB로 전환 준비 |
| 8 | 리뷰별 분석 결과 저장과 식당 단위 분석 통계 구조 확장 |

현재 추천 로직은 MVP 단계에서 rule-based scoring 방식으로 동작한다. 이후 데이터가 충분히 확보되면 리뷰 신뢰도 점수, 태그 신뢰도, 필터 일치도, 리뷰 정보성 등을 함께 반영하는 방식으로 고도화한다.

9.5 Database Development Plan

데이터베이스 개발은 사용자 기능의 핵심 테이블과 관리자 연동 테이블을 구분하여 진행한다. 사용자 기능의 핵심 테이블은 campuses, restaurants, restaurant_summaries, restaurant_tags이며, 이 구조는 캠퍼스 기반 탐색, 지도 및 리스트 표시, 식당 상세 조회, 필터 기반 추천을 제공하기 위한 기본 데이터 구조이다.

현재 구현에서는 관리자 리뷰 CSV 업로드와 분석 작업 상태 관리를 위해 reviews와 analysis_jobs 테이블을 함께 사용한다. 리뷰별 세부 분석 결과, 식당 단위 분석 통계, 다중 관리자 계정, 관리자 변경 이력은 향후 확장 단계에서 review_analyses, restaurant_analyses, admin_users, admin_change_logs 테이블로 분리한다.

Phase	Task
MVP Core	캠퍼스, 식당, 요약, 태그 테이블 구현
MVP Core	CSV import를 통한 식당, 요약, 태그 데이터 적재
MVP Core	campus_slug와 external_id 기반 연결 검증
Current Admin Integration	리뷰 원문 저장용 reviews 테이블 사용
Current Admin Integration	분석 작업 상태 관리를 위한 analysis_jobs 테이블 사용
Expansion	리뷰별 분석 결과용 review_analyses 테이블 추가

Expansion	식당 단위 분석 결과용 restaurant_analyses 테이블 추가
Expansion	관리자 계정 및 변경 이력 테이블 추가

개발 단계에서는 SQLite를 사용하지만, 최종 구조에서는 PostgreSQL로 전환한다. 이를 위해 DB 접속 정보는 코드에 직접 작성하지 않고 환경 변수로 관리한다.

9.6 LLM/Data Development Plan

LLM/Data 파트는 리뷰 텍스트를 분석하여 리뷰 신뢰도 점수, 식당별 요약, 태그 정보를 생성하는 역할을 한다. 본 시스템은 사용자 요청 시점마다 LLM을 실시간으로 호출하지 않고, 사전에 생성된 LLM/Data 결과를 백엔드가 import하여 데이터베이스에 저장한 뒤 사용자 요청 시 조회하는 방식을 기본으로 한다.

9.6.1 Model and Scoring Overview

리뷰 신뢰도 점수 산출에는 skt/A.X-4.0-Light 기반 LoRA 파인튜닝 모델을 사용한다. 모델은 리뷰 텍스트를 입력으로 받아 해당 리뷰가 이벤트나 협찬성 표현의 영향을 상대적으로 적게 받았을 가능성을 확률값으로 산출한다.

모델이 산출한 확률값은 다음과 같이 100점 기준의 리뷰 신뢰도 점수로 변환한다.

$$\text{ReviewCredibilityScore} = \text{non_sponsored_probability} \times 100$$

이 점수는 리뷰의 실제 진위 여부를 단정하는 값이 아니다. 본 시스템은 해당 점수를 리뷰 이벤트 또는 협찬성 표현의 영향을 상대적으로 적게 받았을 가능성을 나타내는 참고 지표로 사용한다. 따라서 사용자 화면과 문서에서는 “가짜 리뷰”, “조작 리뷰”와 같은 단정적 표현을 사용하지 않는다.

명시적인 리뷰 이벤트 문구가 포함된 경우에는 rule-based calibration을 적용할 수 있다. 예를 들어 “리뷰 이벤트”, “사진 리뷰”, “서비스 제공”, “별점 요청”과 같은 표현이 포함된 리뷰는 이벤트 영향 가능성이 높은 것으로 보고 최종 신뢰도 점수에 보정 규칙을 적용한다.

9.6.2 Data Output and Backend Integration

LLM/Data 파트의 결과물은 백엔드가 import할 수 있는 CSV 또는 JSON 형태로 정리한다. MVP 단계에서 백엔드와 연동되는 주요 산출물은 식당 기본 정보, 식당별 요약 정보, 식당별 태그 정보, 평균 리뷰 신뢰도 점수이다.

LLM/Data 결과는 다음 세 가지 CSV를 중심으로 백엔드에 전달한다.

Output File	Purpose	Backend Target
restaurants.csv	식당 기본 정보 제공	restaurants
restaurant_summaries.csv	식당 요약, 대표 메뉴, 분위기·주차·웨이팅 요약, 평균 리뷰 신뢰도 점수 제공	restaurant_summaries
restaurant_tags.csv	필터 및 추천 계산에 사용할 식당별 태그 제공	restaurant_tags

백엔드는 external_id와 restaurant_external_id를 기준으로 LLM/Data 결과와 내부 식당 데이터를 연결한다. 이를 통해 LLM/Data 파트와 백엔드 파트가 독립적으로 작업하더라도 동일한 식당 데이터를 안정적으로 연결할 수 있다.

향후 확장 단계에서는 리뷰별 상세 점수, 모델 버전, 분석 시각, rule-based calibration 적용 여부 등을 review_analyses 테이블에 저장할 수 있다. 또한 식당 단위의 리뷰 신뢰도 통계는 restaurant_analyses 테이블로 분리하여 관리할 수 있다.

9.6.3 Limitations and Future Improvements

LLM/Data 분석 결과는 모델과 데이터의 한계를 가지므로 절대적인 판단 기준으로 사용하지 않는다. 학습 데이터의 라벨은 리뷰의 실제 진위 여부를 직접 증명하는 값이 아니며, 이벤트 영향 가능성과 비협찬성 가능성을 학습하기 위한 약지도 기준으로 해석한다.

향후에는 사람이 직접 검수한 소규모 Gold Label 데이터셋을 구축하여 모델 평가의 신뢰도를 높일 수 있다. 또한 리뷰 정보성 점수, 모델 버전 관리, 분석 작업 재시도 로직, 식당 단위 집계 방식 고도화를 통해 리뷰 신뢰도 분석 결과의 활용성을 개선할 수 있다.

9.7 Configuration and Environment Variables

본 시스템은 외부 API Key, DB 접속 정보, 모델 경로 등 환경에 따라 달라지는 값을 코드 내부에 직접 작성하지 않는다. 개발 환경과 배포 환경을 분리하기 위해 환경 변수를 사용한다.

주요 환경 변수는 다음과 같다.

Variable	Description
VITE_API_BASE_URL	프론트엔드에서 사용할 백엔드 API 기본 URL
VITE_KAKAO_MAP_KEY	Kakao Map JavaScript API Key
DATABASE_URL	백엔드 DB 접속 URL
JWT_SECRET_KEY	관리자 인증 token 서명용 secret
LLM_BASE_MODEL_NAME	base LLM 모델명
LLM_ADAPTER_PATH	LoRA adapter 경로
MODEL_VERSION	분석 결과에 기록할 모델 버전
ALLOWED_ORIGINS	CORS 허용 origin 목록

Kakao Map JavaScript Key는 프론트엔드에서 사용되므로 완전히 숨길 수 없다. 따라서 허용 도메인 제한을 적용해야 한다. 반면 DB 비밀번호, JWT secret, LLM 모델 경로와 같은 민감한 정보는 백엔드 환경 변수로만 관리한다.

9.8 Deployment Plan

MVP 단계에서는 로컬 개발 환경에서 프론트엔드와 백엔드를 각각 실행하여 기능을 검증한다. 이후 통합 테스트 단계에서는 프론트엔드 빌드 결과와 백엔드 API 서버를 연결하고, PostgreSQL 환경에서 데이터 조회와 추천 기능을 확인한다. 최종 시연 단계에서는 HTTPS, CORS, 환경 변수, Kakao Map API Key 허용 도메인 설정을 점검한다.

Stage	Description
Local Development	Vite와 FastAPI 개발 서버를 각각 실행

Local Integration	프론트엔드에서 로컬 백엔드 API 호출
DB Integration	SQLite에서 PostgreSQL로 전환하여 테스트
Demo Deployment	시연용 서버 또는 클라우드 환경에 배포
Final Deployment	HTTPS, CORS, 환경 변수, DB 백업 설정 적용

배포 시에는 다음 사항을 확인해야 한다.

첫째, 프론트엔드가 올바른 백엔드 API URL을 바라보는지 확인한다.

둘째, Kakao Map API Key의 허용 도메인이 배포 도메인과 일치하는지 확인한다.

셋째, DB 접속 정보가 환경 변수로 안전하게 설정되어 있는지 확인한다.

넷째, LLM adapter 파일이 실제 추론 환경에서 접근 가능한 위치에 존재하는지 확인한다.

다섯째, 관리자 API가 인증 없이 노출되지 않도록 한다.

9.9 Version Control and Collaboration Plan

팀 개발은 GitHub를 기반으로 진행한다. 프론트엔드, 백엔드, LLM/Data 파트는 각각 독립적으로 개발하되, API schema와 CSV import schema는 팀 전체가 공유하여 인터페이스 불일치를 줄인다.

협업 원칙은 다음과 같다.

Rule	Description
Branch Management	기능별 branch를 생성하여 작업
Pull Request	주요 변경사항은 PR을 통해 검토 후 병합
Commit Message	변경 내용을 명확히 알 수 있도록 작성
API Change Notice	API endpoint나 응답 형식 변경 시 팀 전체에 공유
Schema Change Notice	CSV, DB schema 변경 시 프론트엔드와 LLM/Data 파트에 공유

설계 문서 수정 시에는 장별 역할을 유지하고, 같은 내용을 여러 장에서 반복하지 않도록 한다. 특히 API endpoint, DB schema, CSV import schema, LLM/Data 점수 해석 기준이 변경되는 경우에는 관련 장을 함께 수정해야 한다. 문체는 보고서형 문체로 통일하고, “추후 수정 예정”과 같은 메모성 표현은 최종 문서에 남기지 않는다.

9.10 Development Risks and Mitigation

본 시스템 개발 과정에서 예상되는 주요 위험 요소와 대응 방안은 다음과 같다.

Risk	Description	Mitigation
API 불일치	프론트엔드 타입과 백엔드 응답 필드가 다를 수 있음	API schema를 먼저 확정하고 공통 문서로 관리
지도 API 의존성	Kakao Map API 실패 시 지도 기능이 중단될 수 있음	리스트 기반 fallback 제공
데이터 품질 문제	리뷰 데이터나 식당 정보가 부정확할 수 있음	관리자 검토 및 수정 기능 제공
LLM 분석 오류	모델 결과가 항상 정확하지 않을 수 있음	rule-based calibration과 관리자 보정 적용
법적 표현 리스크	리뷰를 가짜로 단정하면 문제가 될 수 있음	중립적 표현과 참고 지표 안내 사용
DB 전환 리스크	SQLite에서 PostgreSQL 전환 시 차이가 발생할 수 있음	SQLAlchemy 기반으로 구현하고 통합 테스트 수행
모델 파일 관리 문제	LoRA adapter weight가 누락될 수 있음	Git LFS 또는 Drive 기반 파일 관리 확인
관리자 보안 문제	관리자 API가 인증 없이 노출될 수 있음	인증 token과 AdminGuard 적용

9.11 Development Milestones

개발 일정은 MVP 완성과 향후 확장을 구분하여 진행한다.

Milestone	Main Tasks
M1. Frontend Prototype	라우팅, 캠퍼스 선택, 지도/리스트 UI, 상세 페이지 구성
M2. Backend MVP API	캠퍼스 조회, 식당 목록 조회, 상세 조회, 추천 API 구현
M3. Data Import	식당, 요약, 태그 CSV import 및 DB 저장
M4. Frontend-Backend Integration	실제 API 연동, 지도 마커와 리스트 데이터 동기화
M5. Recommendation Validation	필터 조건별 추천 점수와 추천 근거 검증
M6. Error/Fallback Handling	지도 실패, 데이터 없음, API 오류 처리
M7. Admin Prototype	관리자 로그인, 식당/리뷰/분석 관리 화면 구성
M8. Analysis Extension	리뷰 업로드, 분석 작업, 리뷰별 신뢰도 저장 구조 확장
M9. Final Testing	통합 테스트, 사용자 테스트, 발표용 시연 준비

Supporting Information

10.1 Objectives

본 장에서는 디자인 명세서의 이해를 돕기 위한 보조 정보를 정리한다. 앞선 장들이 시스템의 아키텍처, 프론트엔드, 백엔드, 프로토콜, 데이터베이스, 테스트, 개발 계획을 설명했다면, 본 장은 문서 전반에서 사용한 주요 용어, 요구사항과 설계 요소의 대응 관계, 향후 확장 가능성, 문서 관리 정보를 정리하는 역할을 한다.

본 장의 주요 목적은 다음과 같다.

첫째, 시스템 설계에서 반복적으로 사용되는 주요 용어를 명확히 정의한다.

둘째, 요구사항 명세서의 주요 기능 요구사항이 디자인 명세서의 어느 부분에 반영되었는지 추적할 수 있도록 한다.

셋째, MVP 범위와 향후 확장 범위를 구분하여 개발 우선순위를 명확히 한다.

넷째, 문서의 유지보수와 변경 관리를 위한 기준을 제공한다.

10.2 Glossary

Term	Description
SKKU Gourmap	성균관대학교 캠퍼스 주변 식당을 추천하는 본 프로젝트의 서비스명
Campus Slug	프론트엔드 라우팅과 백엔드 API에서 캠퍼스를 식별하는 문자열 값. 예: natural, humanities
Restaurant External ID	LLM/Data 결과와 백엔드 DB의 식당 데이터를 연결하기 위한 외부 식당 식별자

Review Credibility Score	리뷰가 이벤트나 협찬성 표현의 영향을 상대적으로 적게 받았을 가능성을 나타내는 참고 점수
Average Trust Score	식당별 리뷰 신뢰도 점수의 평균값
Recommendation Score	필터 조건, 식당 태그, 리뷰 신뢰도, 요약 정보 등을 반영하여 계산한 식당 추천 점수
Recommendation Reasons	특정 식당이 추천된 이유를 사용자가 이해할 수 있도록 제공하는 설명 문장
Restaurant Summary	리뷰 분석 결과를 바탕으로 생성된 식당 단위 요약 정보
Restaurant Tag	필터 검색과 추천 점수 계산에 사용되는 식당 속성 태그
Rule-based Calibration	명시적 이벤트 문구가 포함된 리뷰의 신뢰도 점수를 보정하는 후처리 규칙
Admin Guard	인증된 관리자만 관리자 화면에 접근할 수 있도록 제한하는 프론트엔드 보호 로직
Analysis Job	리뷰 분석 작업의 생성, 실행, 완료, 실패 상태를 관리하는 작업 단위
MVP	Minimum Viable Product. 초기 시연과 검증을 위한 최소 기능 제품

10.3 Requirement Traceability

본 절에서는 주요 기능 요구사항이 디자인 명세서의 어떤 설계 요소에 반영되었는지 정리한다.

Requirement	Description	Related Design Sections
FR-01	캠퍼스 선택	3.3 Use Case Diagram, 4.3 Route Design, 6.5 Request Parameter Design
FR-02	캠퍼스별 식당 목록 조회	4.4 Page Design, 5.3 Backend

		Components, 6.6 Restaurant List API
FR-03	지도 기반 식당 표시	4.7 Kakao Map Integration Design, 8.3 Frontend Testing
FR-04	리스트 기반 식당 표시	4.4 Page Design, 4.9 Error and Fallback Design, 8.3 Frontend Testing
FR-05	식당 상세 정보 조회	4.4 Page Design, 5.3 Backend Components, 6.8 Restaurant Detail API
FR-06	필터 조건 입력	4.5 Component Design, 4.8 State Management Design, 6.7 Recommendation API
FR-07	필터 기반 추천 점수 계산	5.3 Backend Components, 6.7 Recommendation API, 8.6 Recommendation Testing
FR-08	추천 근거 제공	4.4 Page Design, 5.3 Backend Components, 6.7 Recommendation API
FR-09	리뷰 신뢰도 분석	5.3 Backend Components, 7.9 Future Schema Extension
FR-10	식당별 요약 생성	5.3 Backend Components, 7.4 Table Descriptions, 7.6 LLM/Data Import Schema
FR-11	식당 태그 생성	5.3 Backend Components, 7.4 Table Descriptions, 7.6 LLM/Data Import Schema
FR-12	관리자 인증	4.4 Page Design, 5.3 Backend Components, 6.11 Authentication Protocol
FR-13	식당 정보 관리	5.3 Backend Components, 7.4 Table Descriptions, 10.4 Current Implementation Scope Summary

FR-14	리뷰 데이터 관리	5.3 Backend Components, 7.4 Table Descriptions, 8.8 Admin and Security Testing
FR-15	분석 작업 상태 관리	5.3 Backend Components, 7.9 Future Schema Extension, 8.8 Admin and Security Testing
FR-16	예외 상황 처리	4.9 Error and Fallback Design, 6.12 Error Response Design, 8.4 Backend API Testing

사용자용 MVP는 FR-01부터 FR-08, FR-16을 중심으로 검증한다. 현재 구현에는 관리자 인증, 식당 정보 관리, 리뷰 데이터 관리, 분석 작업 상태 관리 기능이 포함된다. 다만 세부 분석 결과 저장, 다중 관리자 계정, 권한 분리, 변경 이력 관리는 향후 확장 항목으로 둔다.

10.4 Current Implementation Scope Summary

본 절은 사용자용 MVP 기능과 현재 관리자 연동 구현 범위를 함께 요약한다.

Area	Included Scope
Campus Selection	자연과학캠퍼스, 인문사회캠퍼스 선택
Restaurant Browsing	캠퍼스별 식당 목록 조회
Map UI	Kakao Map 기반 식당 위치 표시
List UI	식당 카드 및 리스트 표시
Restaurant Detail	식당 기본 정보, 요약, 태그, 평균 신뢰도 점수 표시
Recommendation	필터 조건 기반 rule-based 추천
Data Import	식당, 요약, 태그 CSV import
Admin Login	환경 변수 기반 단일 관리자 로그인 및 JWT access token 발급

Admin Restaurant Management	관리자 식당 목록 조회, 식당 추가, 수정, soft delete
Review Upload	관리자 CSV 기반 리뷰 원문 업로드 및 restaurant_external_id 매칭
Analysis Job Management	분석 작업 생성, 목록 조회, 단건 상태 조회 및 progress/status 반환
Error Handling	데이터 없음, 지도 실패, API 오류 안내
Neutral Expression	리뷰 진위 여부를 단정하지 않는 표현 사용

MVP에서 제외하거나 제한적으로 구현하는 기능은 다음과 같다.

Area	Not Included or Planned
Real-time GPS	사용자의 실시간 위치 기반 추천은 제외
Nationwide Service	전국 단위 맛집 추천은 제외
Reservation/Payment	예약, 결제, 쿠폰 기능은 제외
Real-time Waiting	실시간 웨이팅 정보 제공은 제외
Personal Account	개인 계정 기반 취향 학습은 제외
Full Admin Account System	다중 관리자 계정, 역할 기반 권한 분리, 관리자 변경 이력 저장은 향후 확장
Real-time LLM Inference	사용자 요청 시점의 실시간 LLM 호출은 제외
Review-level Display	개별 리뷰 원문 전체 노출은 제한

10.5 Future Extension Items

향후 시스템 확장 시 고려할 수 있는 기능은 다음과 같다.

첫째, 관리자 기능 고도화이다. 현재 구현된 단일 관리자 인증, 식당 정보 관리, 리뷰 CSV 업로드, 분석 작업 생성 및 상태 조회 기능을 기반으로, 향후 다중 관리자 계정, 역할 기반 권한 분리, 관리자 변경 이력 저장 기능을 추가할 수 있다.

둘째, 리뷰 분석 자동화이다. 현재 구현에서는 리뷰 원문 저장과 분석 작업 상태 관리를 우선 제공한다. 향후에는 분석 작업이 실제 LLM/Data Analysis Module과 연동되어 리뷰별 신뢰도 점수, 모델 버전, 분석 시각을 자동으로 저장하도록 확장할 수 있다.

셋째, 데이터베이스 확장이다. 향후 review_analyses, restaurant_analyses, admin_users, admin_change_logs 테이블을 추가하여 리뷰별 분석 결과, 식당 단위 분석 통계, 다중 관리자 계정, 관리자 변경 이력을 체계적으로 관리할 수 있다.

넷째, 추천 로직 고도화이다. 현재 구현은 rule-based scoring을 사용하지만, 향후에는 식당 단위 리뷰 신뢰도 통계, 태그 신뢰도, 사용자 선택 필터 가중치, 사용자 피드백을 더 정교하게 반영할 수 있다.

다섯째, UI/UX 개선이다. 모바일 환경에서 리스트 중심 탐색을 강화하고, 지도와 리스트 간 상호작용을 개선하며, 추천 근거와 리뷰 신뢰도 점수의 의미를 더 직관적으로 안내할 수 있다.

여섯째, 운영 안정성 개선이다. PostgreSQL 전환, 배포 환경 분리, HTTPS 적용, API versioning, 로그 관리, 분석 작업 재시도, 모델 버전 관리를 추가할 수 있다.

10.6 Legal and Ethical Considerations

본 시스템은 리뷰 분석 결과를 사용자에게 제공하지만, 리뷰의 실제 진위 여부를 단정하지 않는다. 리뷰 신뢰도 점수는 공개 리뷰 데이터와 모델 분석 결과를 바탕으로 산출된 참고 지표이며, 특정 리뷰나 식당을 "가짜", "조작", "불신 대상"으로 판단하기 위한 값이 아니다.

따라서 시스템은 다음 원칙을 따른다.

첫째, 사용자 화면과 문서에서 "가짜 리뷰", "조작 리뷰", "불신 식당"과 같은 단정적 표현을 사용하지 않는다.

둘째, "리뷰 신뢰도 점수", "비협찬성 가능성", "이벤트 영향 가능성", "리뷰 데이터 기반 참고 지표"와 같은 중립적 표현을 사용한다.

셋째, 원본 리뷰 전체를 무단으로 재게시하지 않고, 요약, 태그, 점수, 핵심 포인트 중심으로 제공한다.

넷째, 리뷰 작성자 식별 정보, 닉네임, 프로필 이미지, 전화번호 등 개인정보는 사용자 화면에 노출하지 않는다.

다섯째, LLM 분석 결과가 부정확할 수 있으므로 관리자가 요약과 태그를 검토하고 수정할 수 있는 구조를 둔다.

여섯째, 사용자에게 리뷰 신뢰도 점수의 의미와 한계를 명확히 안내한다.

10.7 Document Change Management

본 디자인 명세서는 개발 진행 상황에 따라 지속적으로 수정될 수 있다. 특히 API endpoint, 데이터베이스 schema, 추천 점수 계산 방식, LLM 분석 결과 구조, 관리자 인증 방식은 구현 과정에서 변경될 가능성이 있으므로 변경 이력을 관리해야 한다.

문서 변경 시에는 다음 원칙을 따른다.

Change Target	Management Rule
API 변경	프론트엔드와 백엔드 담당자 모두에게 공유
DB schema 변경	import script, API 응답, 프론트엔드 타입에 영향 여부 확인
추천 로직 변경	테스트 케이스와 추천 근거 출력 결과 함께 수정
LLM 모델 변경	모델 버전과 score 해석 기준 업데이트
UI 변경	요구사항 대응 관계와 테스트 항목 함께 수정
관리자 기능 변경	인증, 권한, 변경 이력 관리 기준 업데이트

문서의 각 장은 다음과 같은 기준으로 업데이트한다.

Chapter	Update Trigger
---------	----------------

3. Overall Architecture	시스템 구성 요소 또는 모듈 관계 변경
4. Frontend Architecture	라우트, 화면 구조, 컴포넌트 구조 변경
5. Backend Architecture	API 서버 구조, service layer, 분석 모듈 변경
6. Protocol Design	API endpoint, request/response schema 변경
7. Database Design	테이블, 컬럼, 관계, index 변경
8. Testing Plan	기능 추가, 오류 처리 방식 변경
9. Development Plan	개발 환경, 배포 방식, 구현 우선순위 변경

10.8 Document Revision History

Version	Date	Description	Author
v1.0	2026-05	Initial draft of the software design specification	Team 1
v1.1	2026-05	Updated frontend, backend, protocol, and database sections based on current implementation	Team 1
v1.2	2026-05	Revised MVP, current implementation, and future extension scope for consistency across chapters	Team 1
v1.3	2026-05	Reduced duplicated content and aligned terminology such as LLM/Data Analysis Module, Review Credibility Score, Average Trust Score, and Recommendation Score	Team 1

10.9 Reference Materials

본 디자인 명세서는 요구사항 명세서, 프론트엔드 및 백엔드 소스 코드, 데이터베이스 schema 문서, LLM/Data 분석 설계 자료, Kakao Map API 문서, FastAPI 및 SQLAlchemy 공식 문서를 바탕으로 작성되었다. 상세 참고 자료는 2.2.8 References에 정리한다.